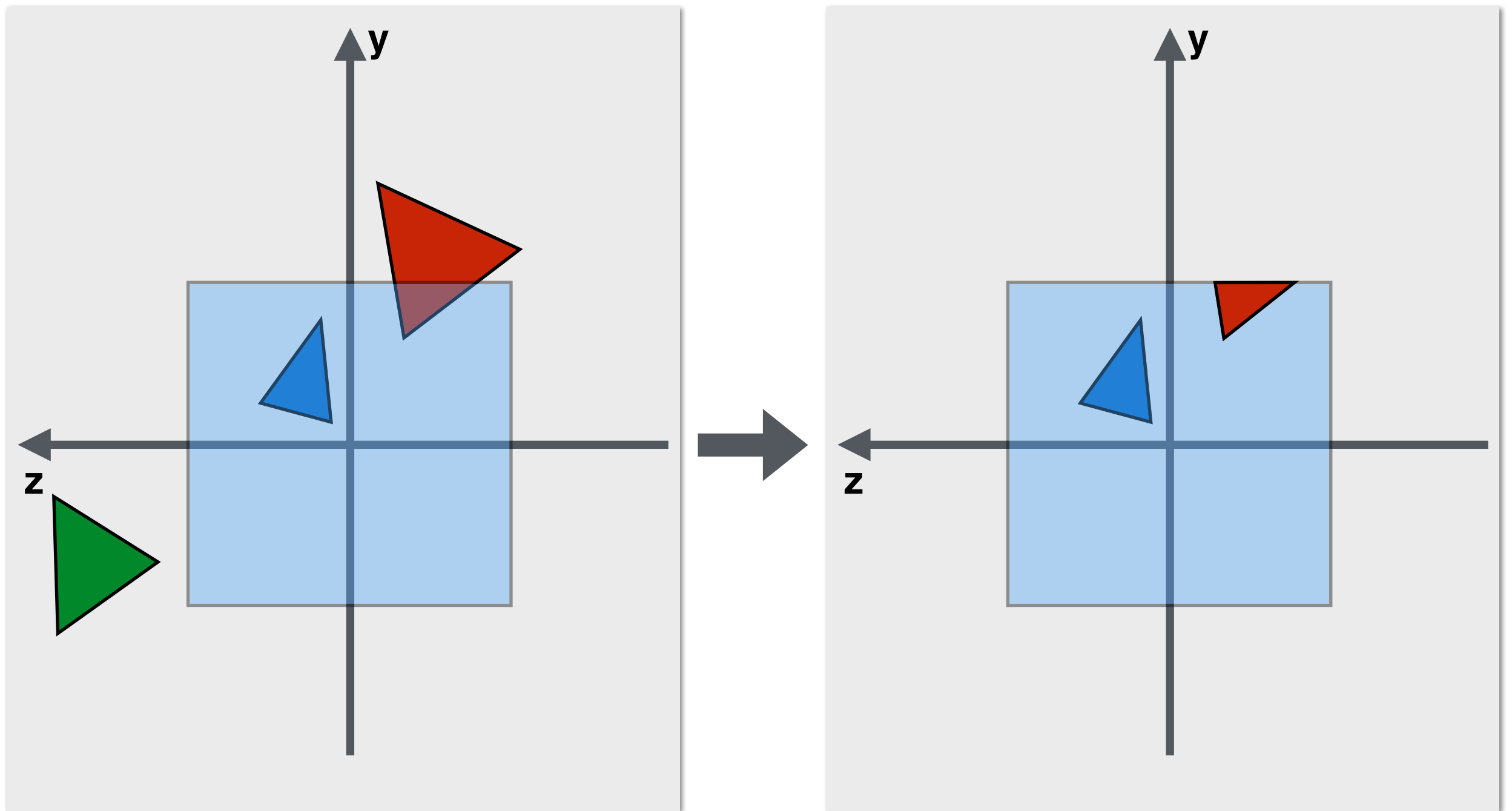


Decupare

Decuppare



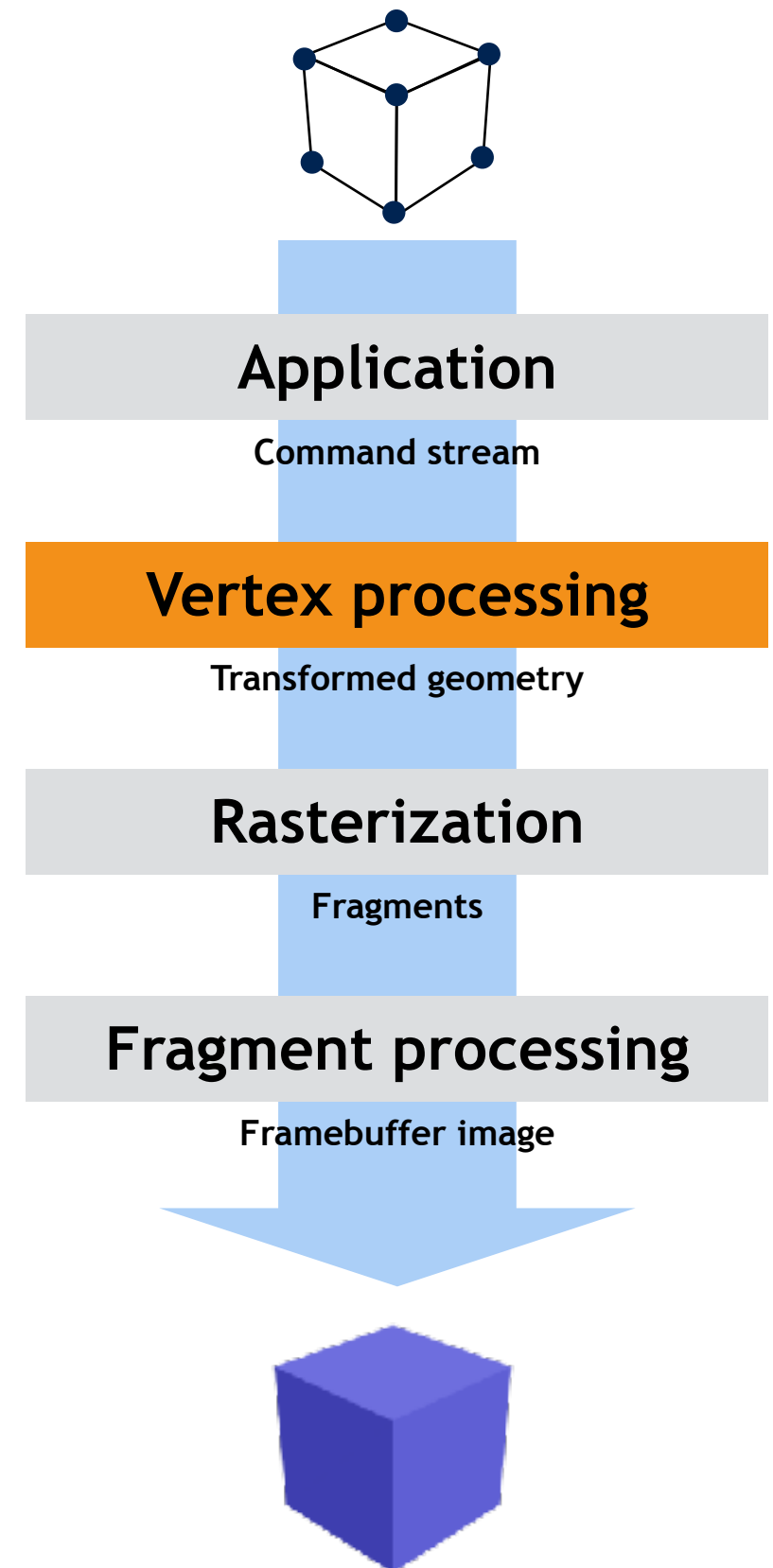
Decupare

Evitarea desenării porțiunilor din primitivele grafice care se găsesc în exteriorul zonei vizibile

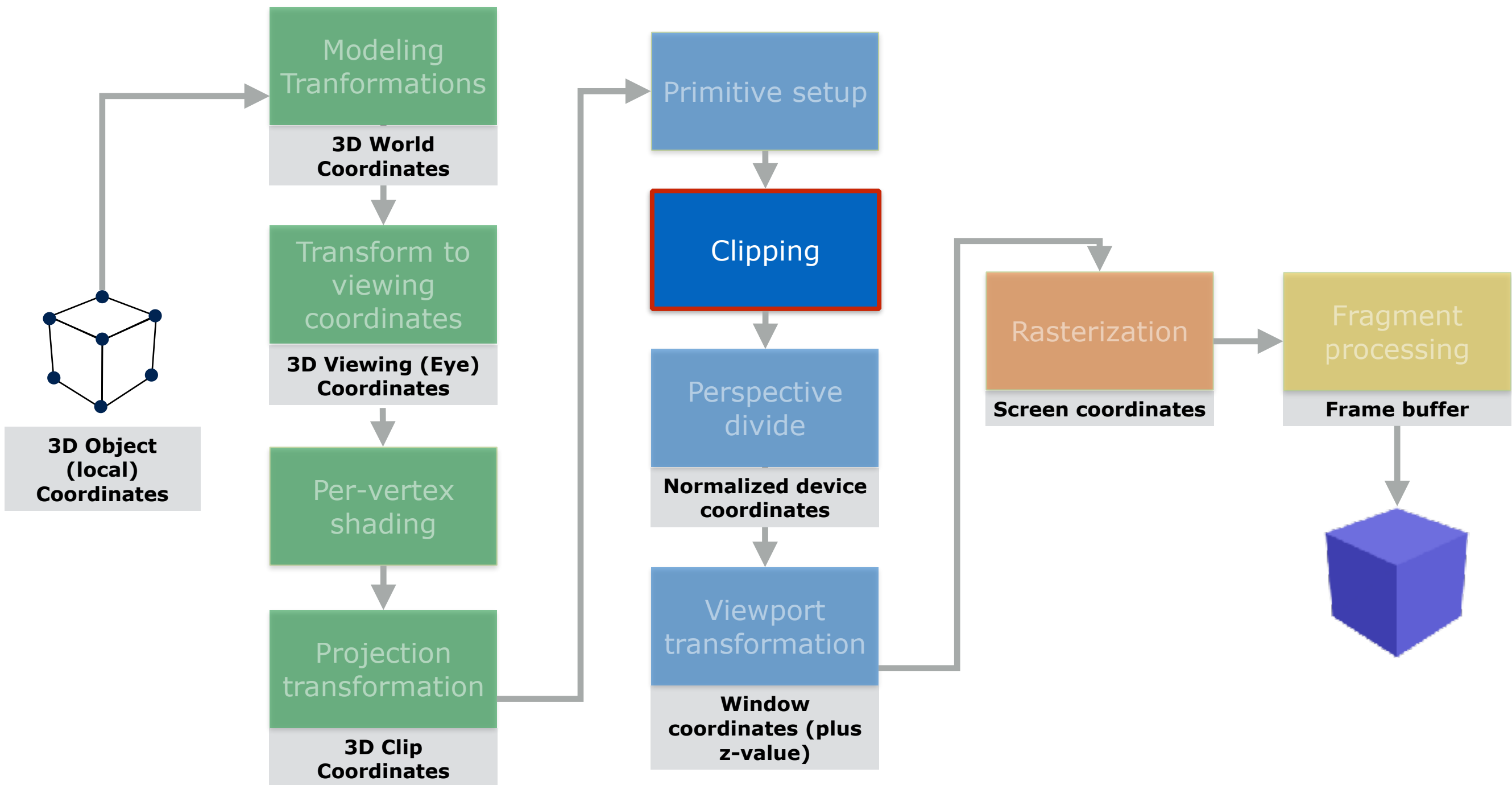
Operația de decupare se realizează:

- în coordonate reale folosind cele 6 planuri care definesc volumul de vizualizare
- în spațiul 4D transformat înainte de operația de **homogeneous (perspective) divide**

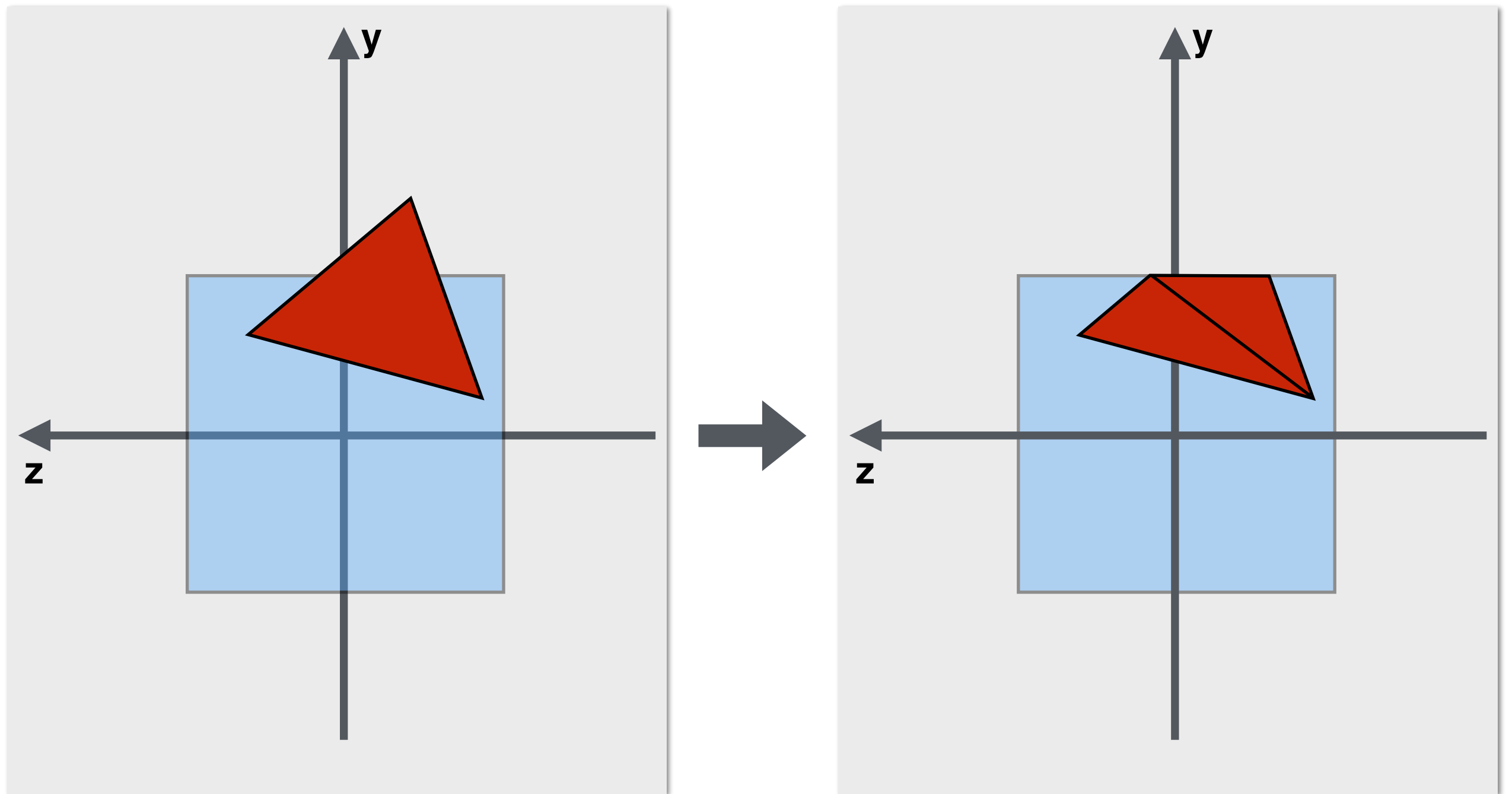
```
for each of six planes do
  if (triangle entirely outside of plane) then
    break (triangle is not visible)
  else if triangle spans plane then
    clip triangle
  if (quadrilateral is left) then
    break into two triangles
```



Decupare



Decupare



Decupare

Frustum culling

- Eliminarea poligoanelor care sunt în întregime în exteriorul volumului de vizualizare

Near-plane clipping

- Decuparea poligonelor față de planul apropiat (**near**)

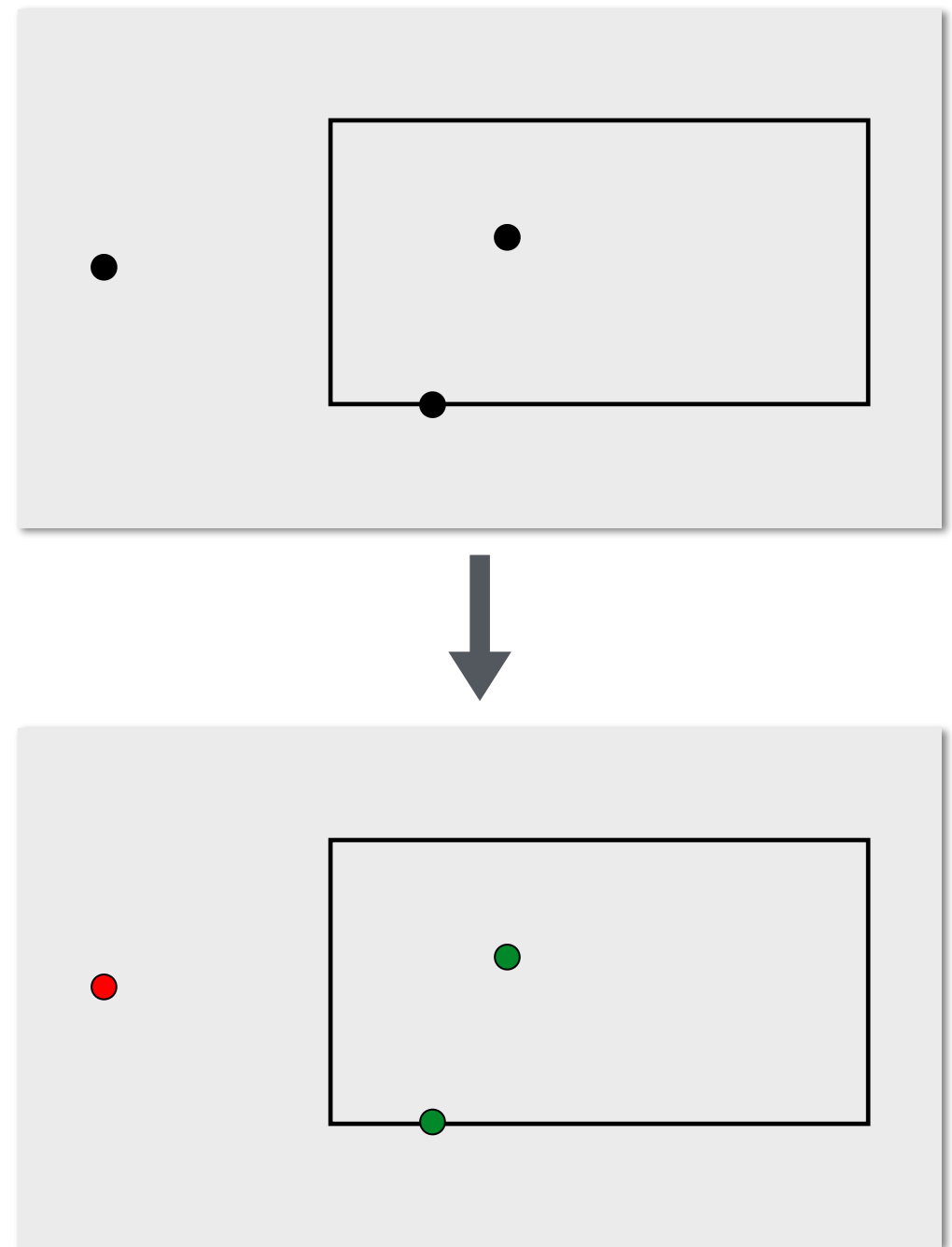
Whole-frustum clipping

- Decuparea poligonelor față de celelalte planuri, laterale și planul depărtat (**far**)

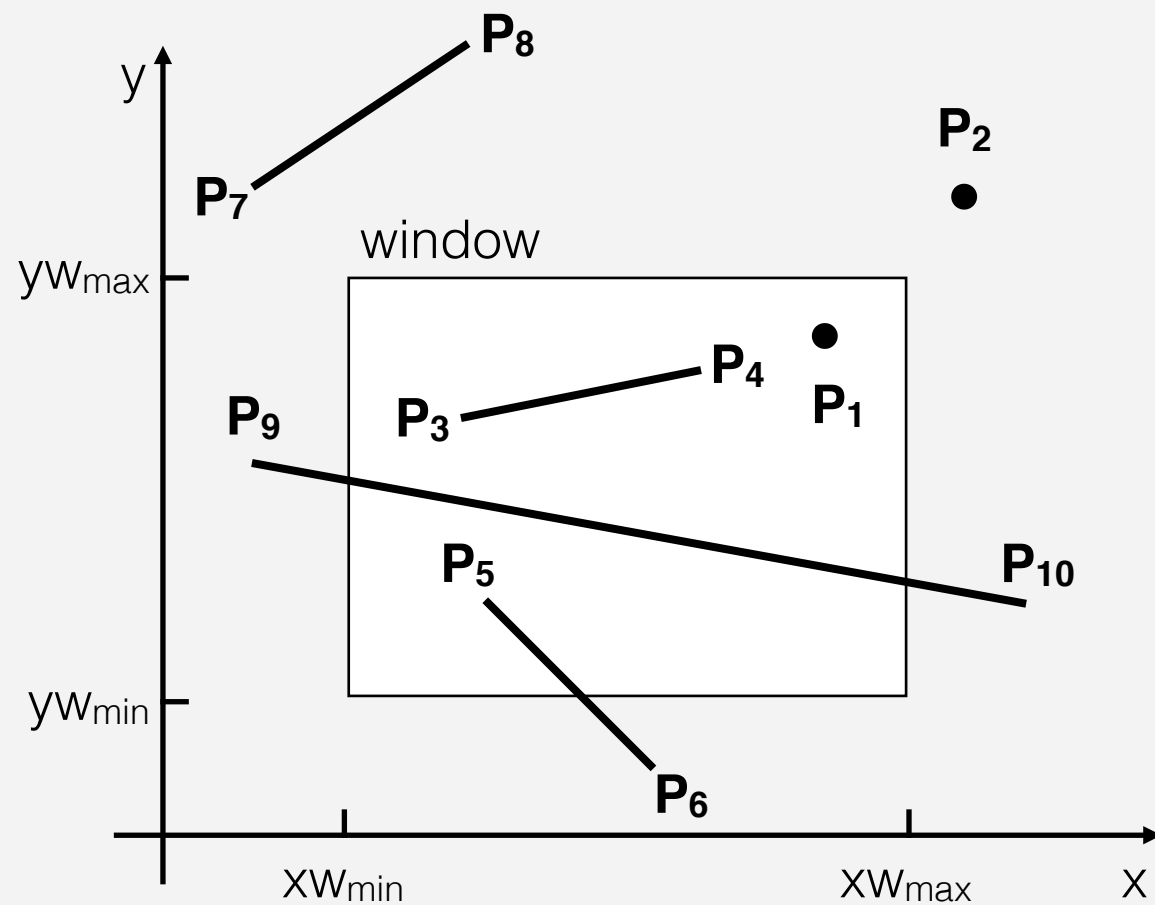
Decupareaa punctelor

Test de incluziune:

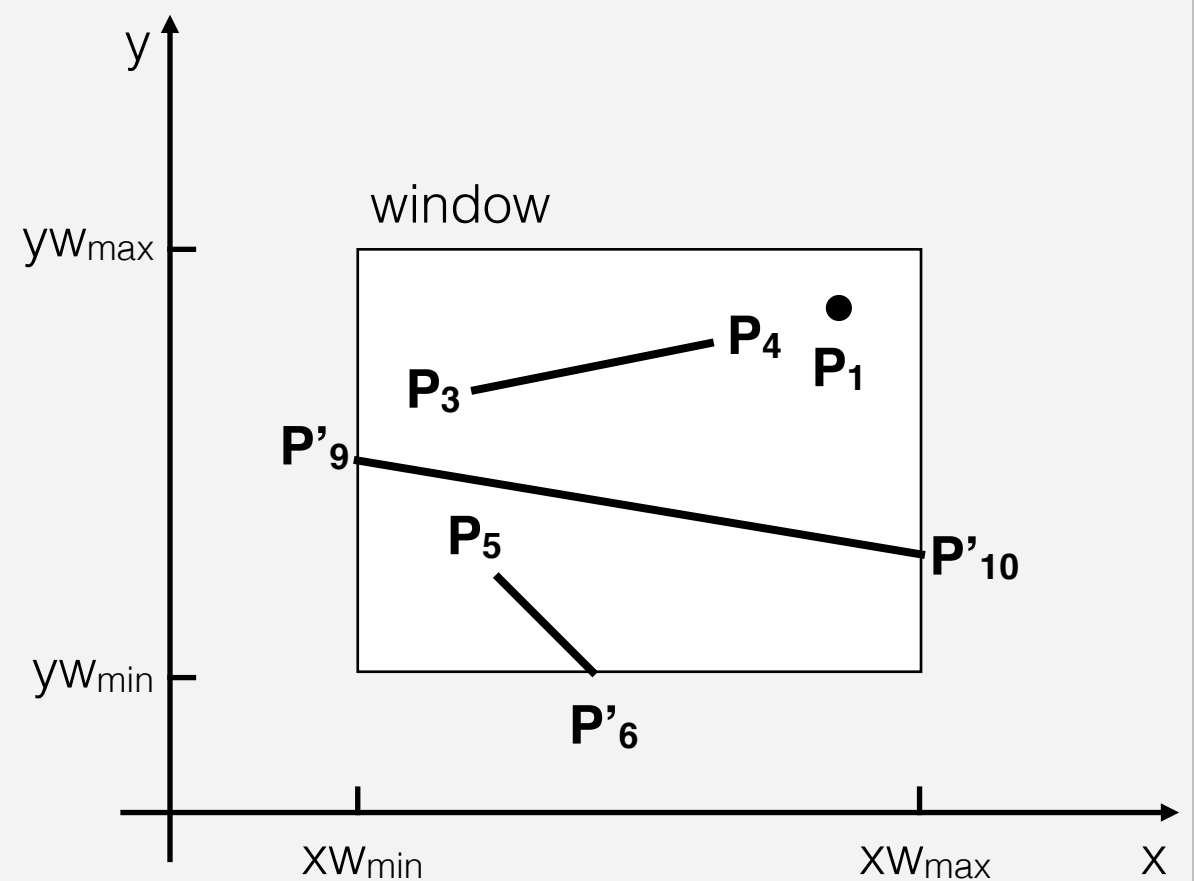
- $x_{\min} \leq x \leq x_{\max}$
- $y_{\min} \leq y \leq y_{\max}$



Decuparea dreptelor



Înainte de decupare



După decupare

Algoritmul Cohen-Sutherland

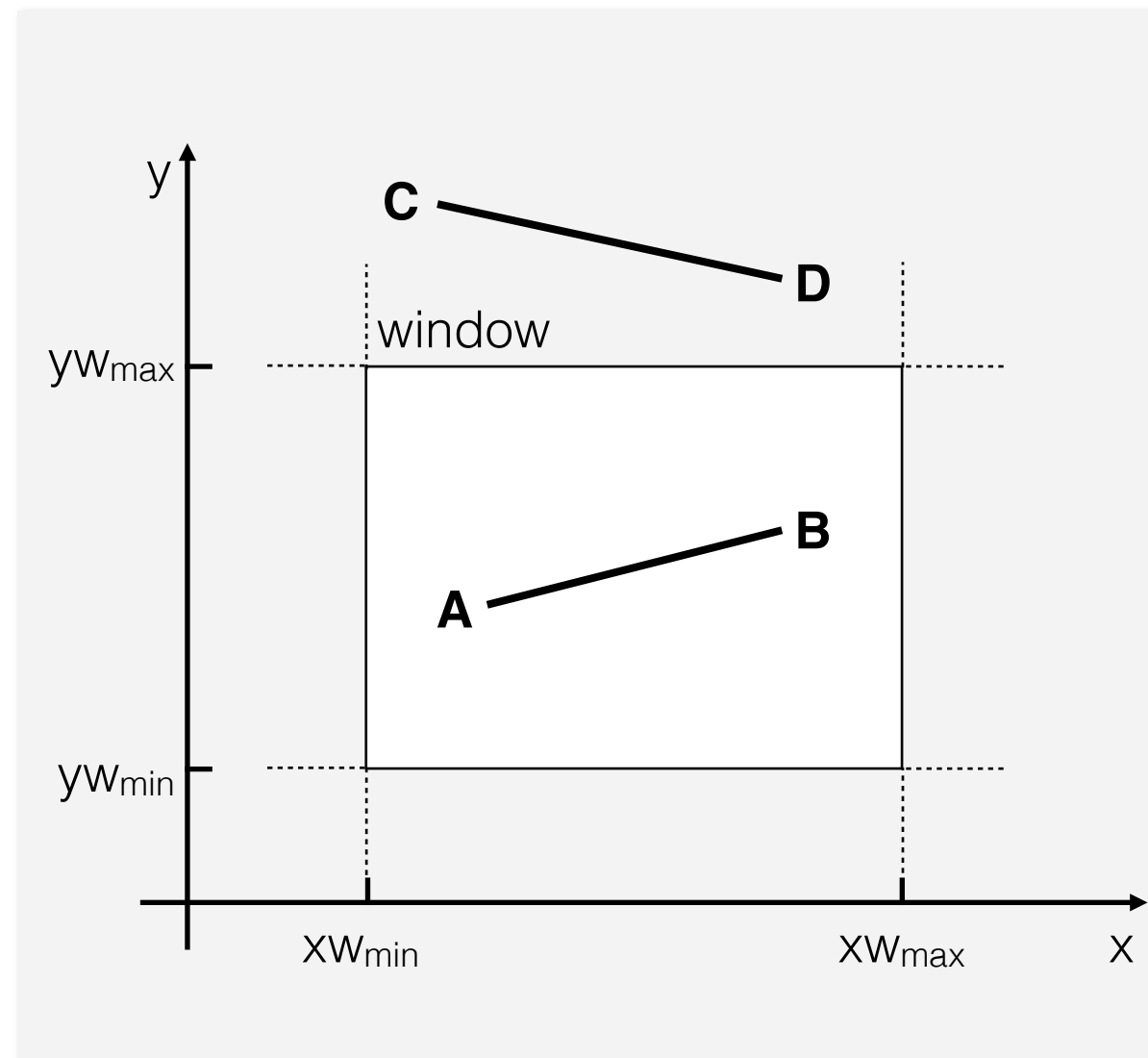
Atribuire a unui cod pentru fiecare punct care definește segmentul de decupat

Decupare iterativă a segmentului de dreaptă astfel încât să fie în întregime în interiorul ferestrei sau în întregime în exteriorul ferestrei

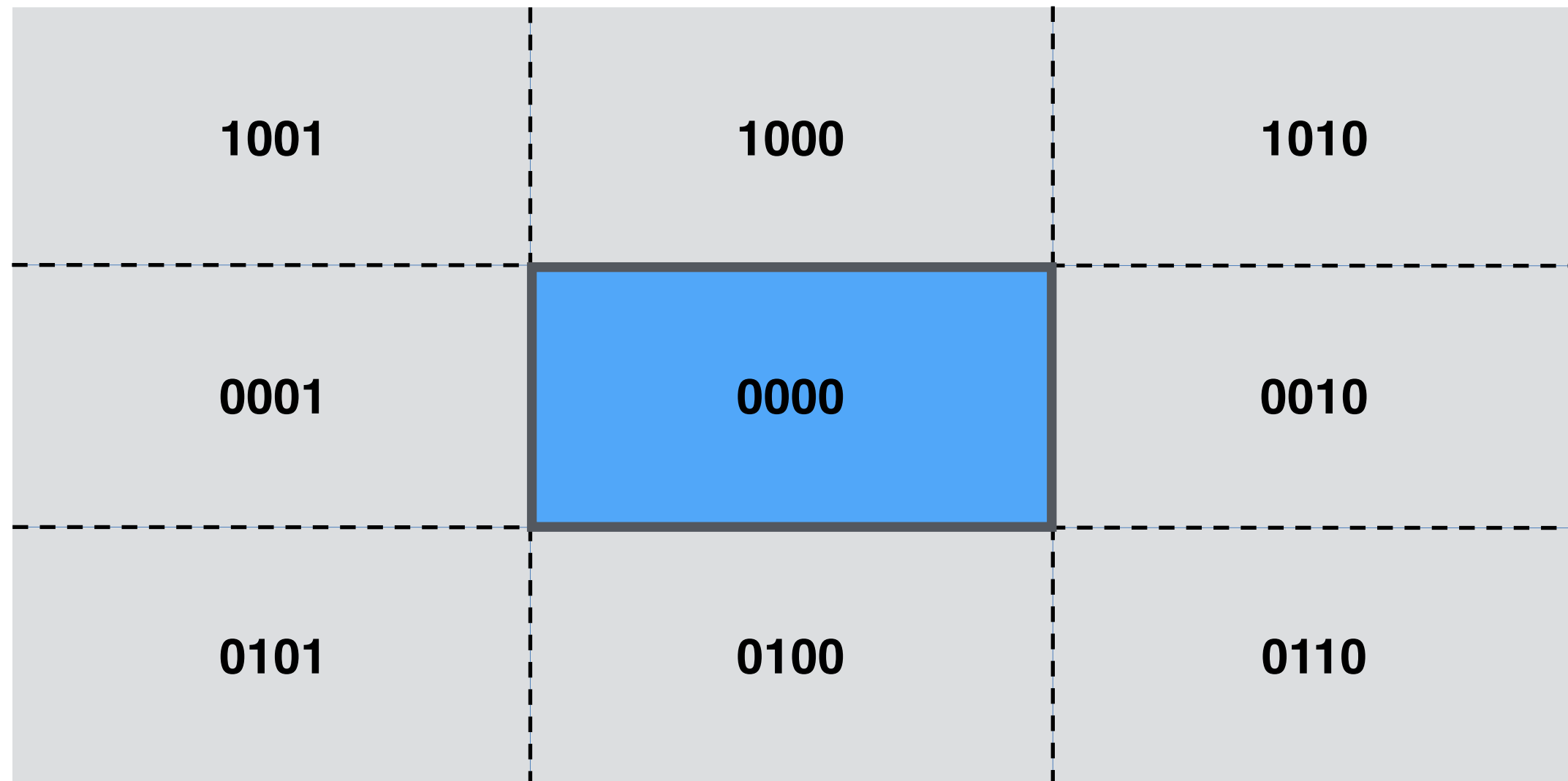
Cazuri simple

Acceptare (AB)

Respingere (CD)



Algoritmul Cohen-Sutherland



Region outcodes:

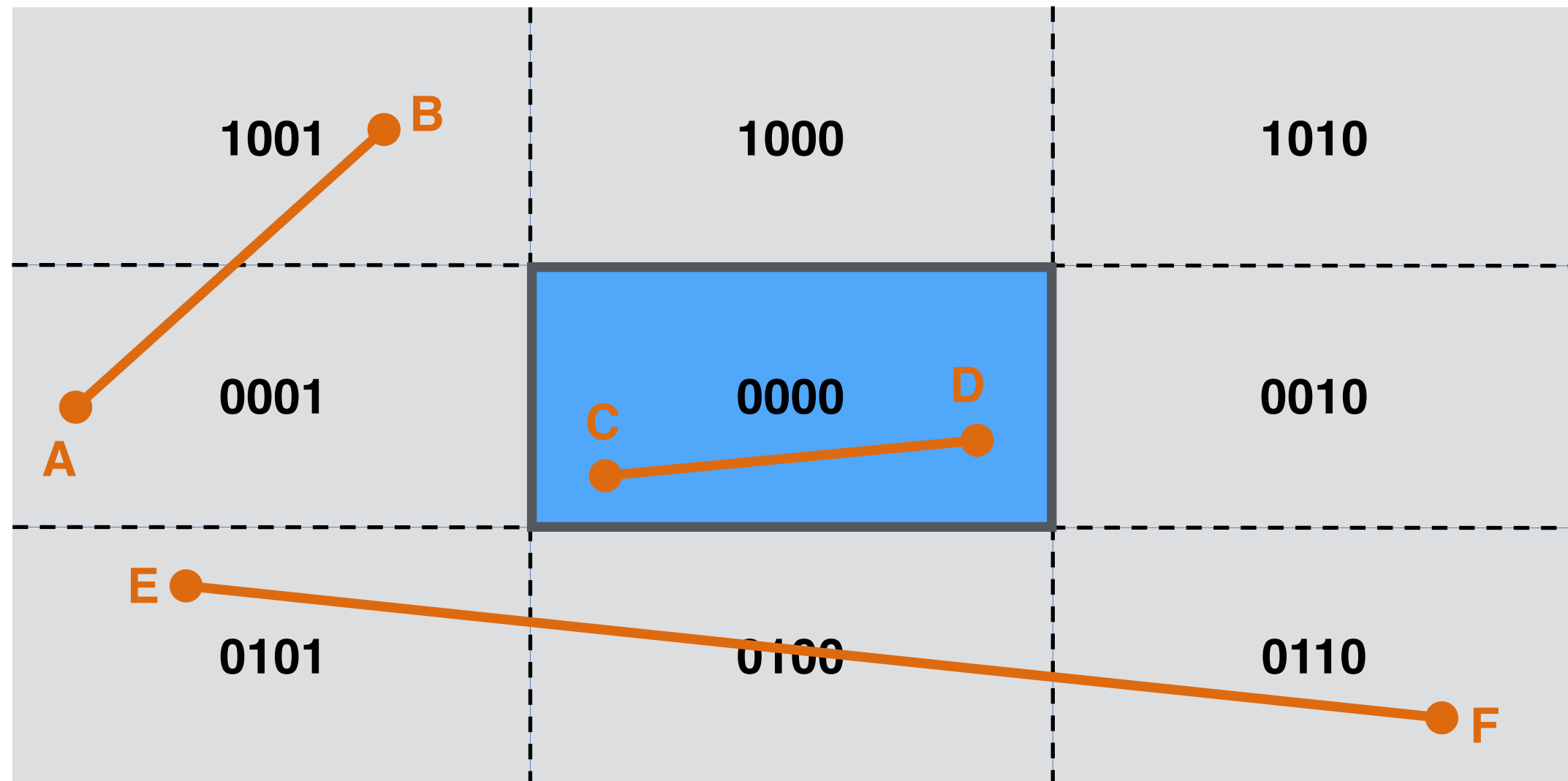
Above

Below

Right

Left

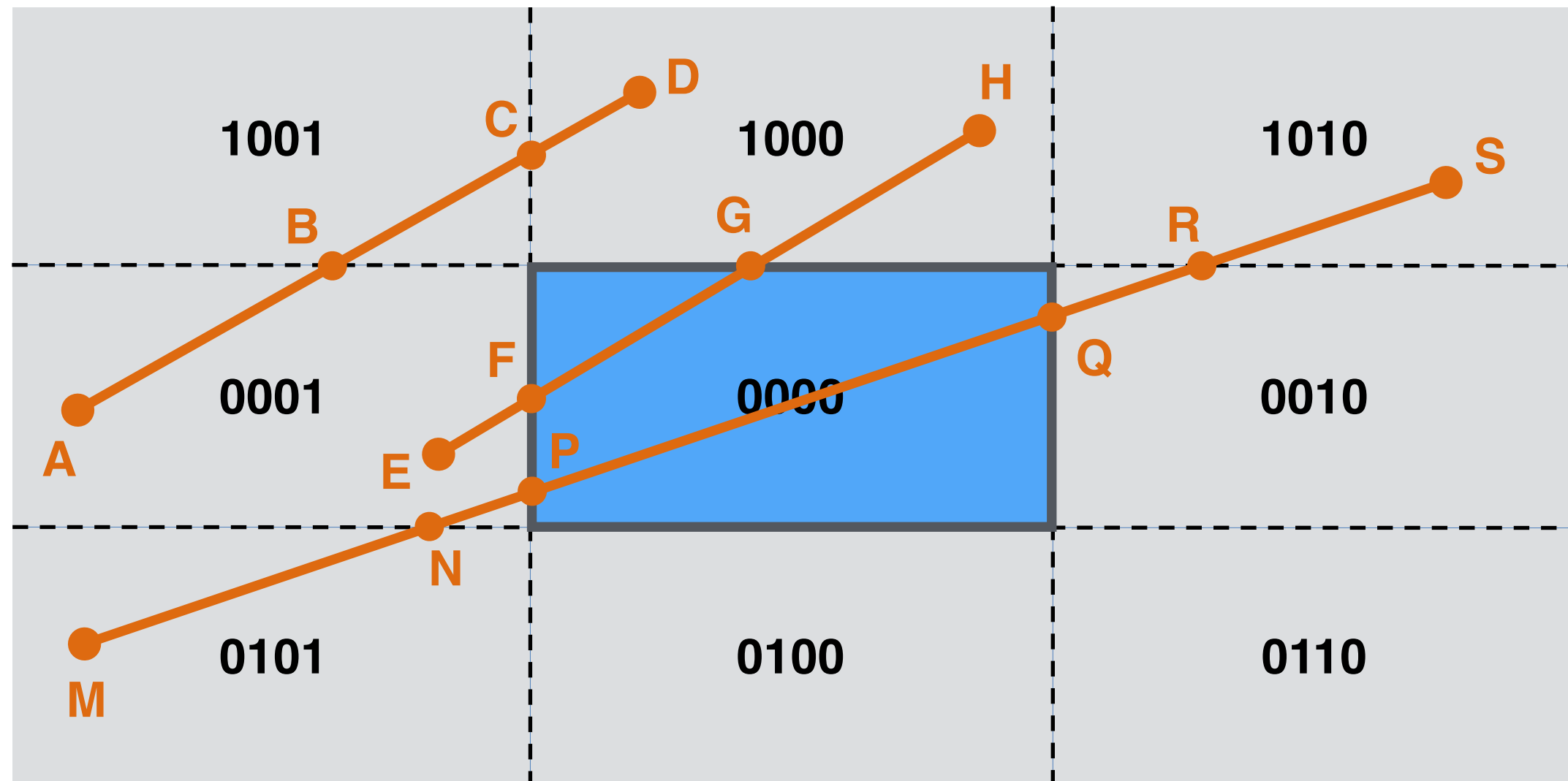
Algoritmul Cohen-Sutherland



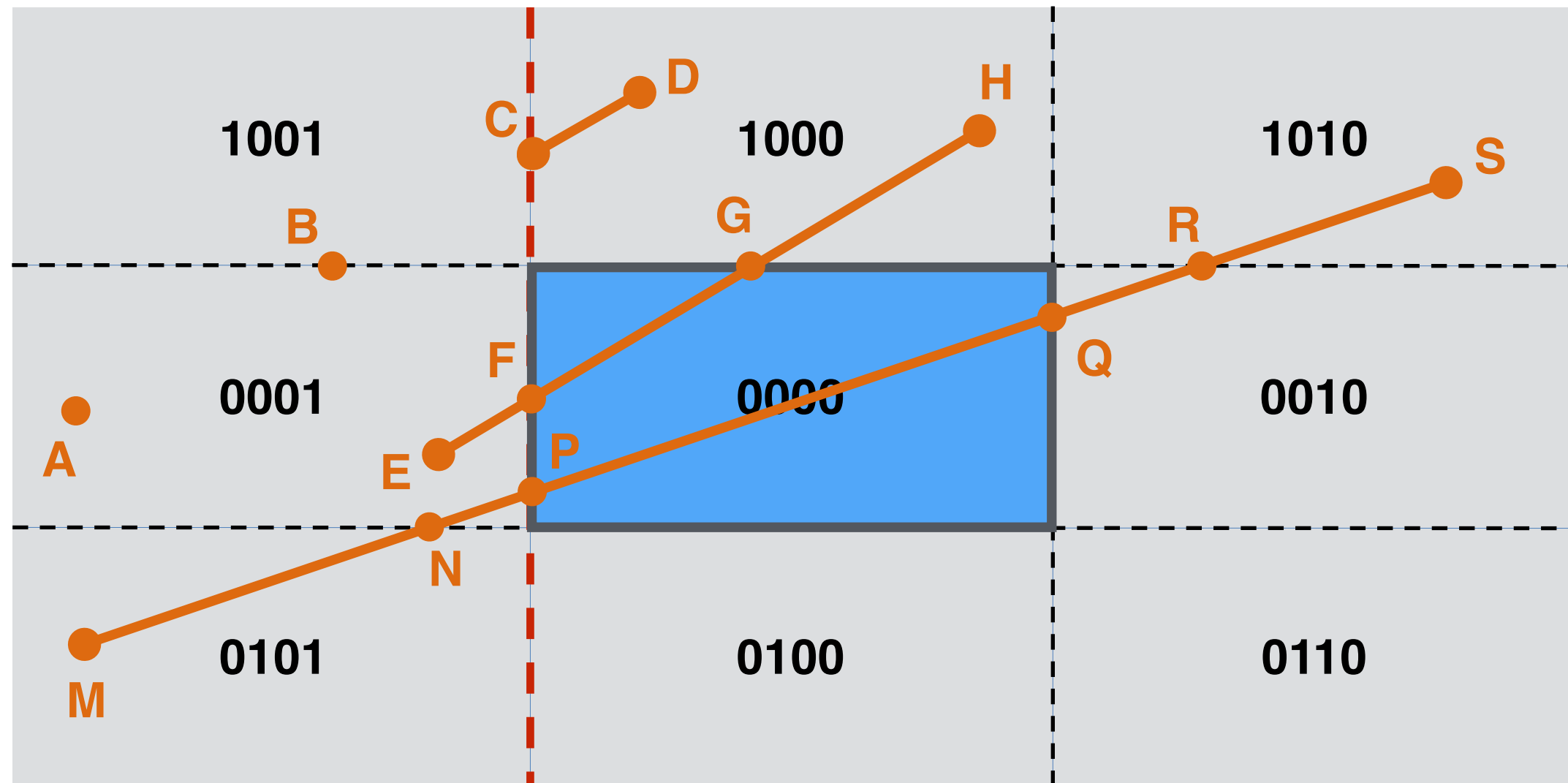
Acceptare simplă: **CD**

Respingere simplă: **AB, EF**

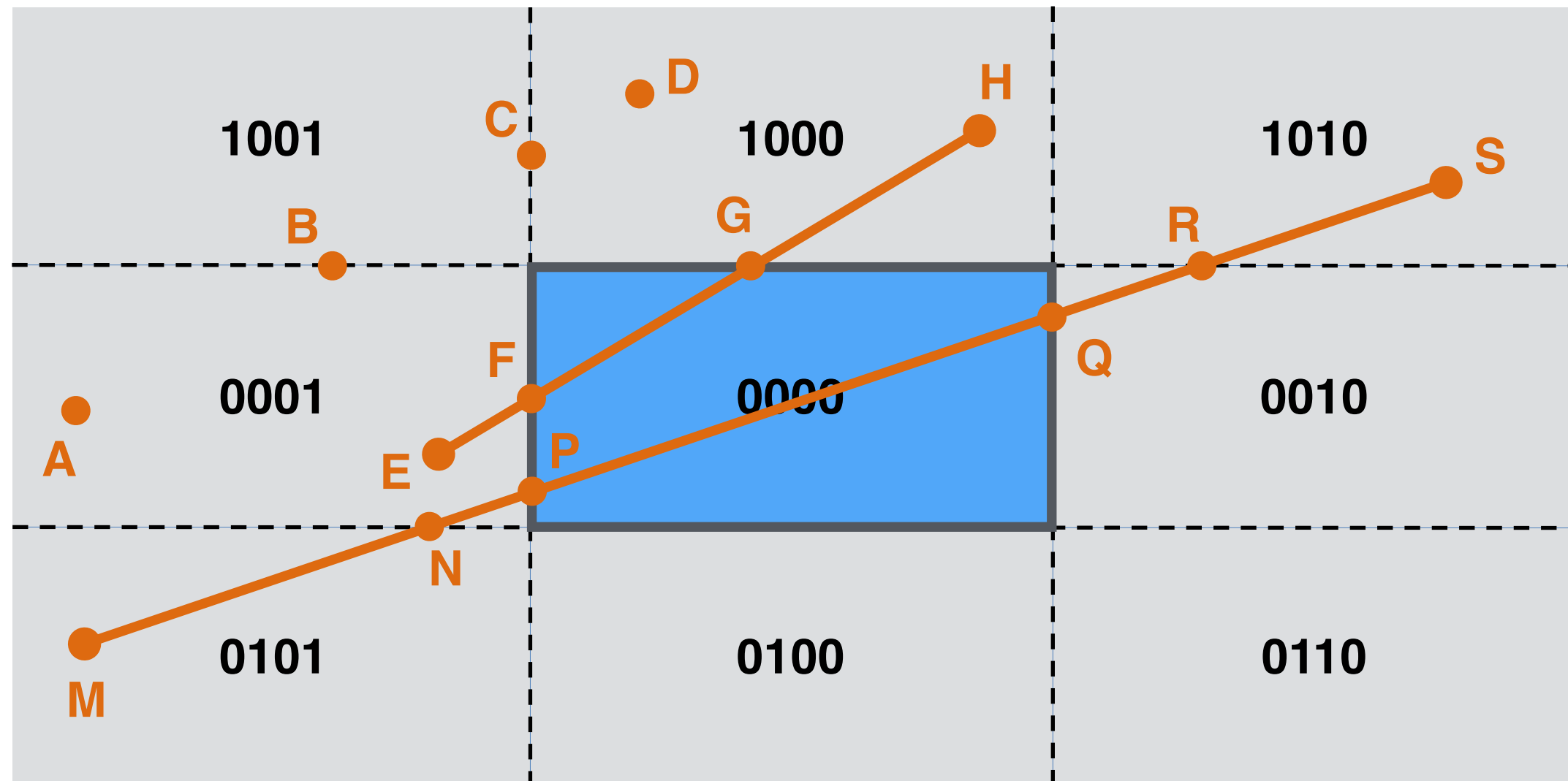
Algorithmul Cohen-Sutherland



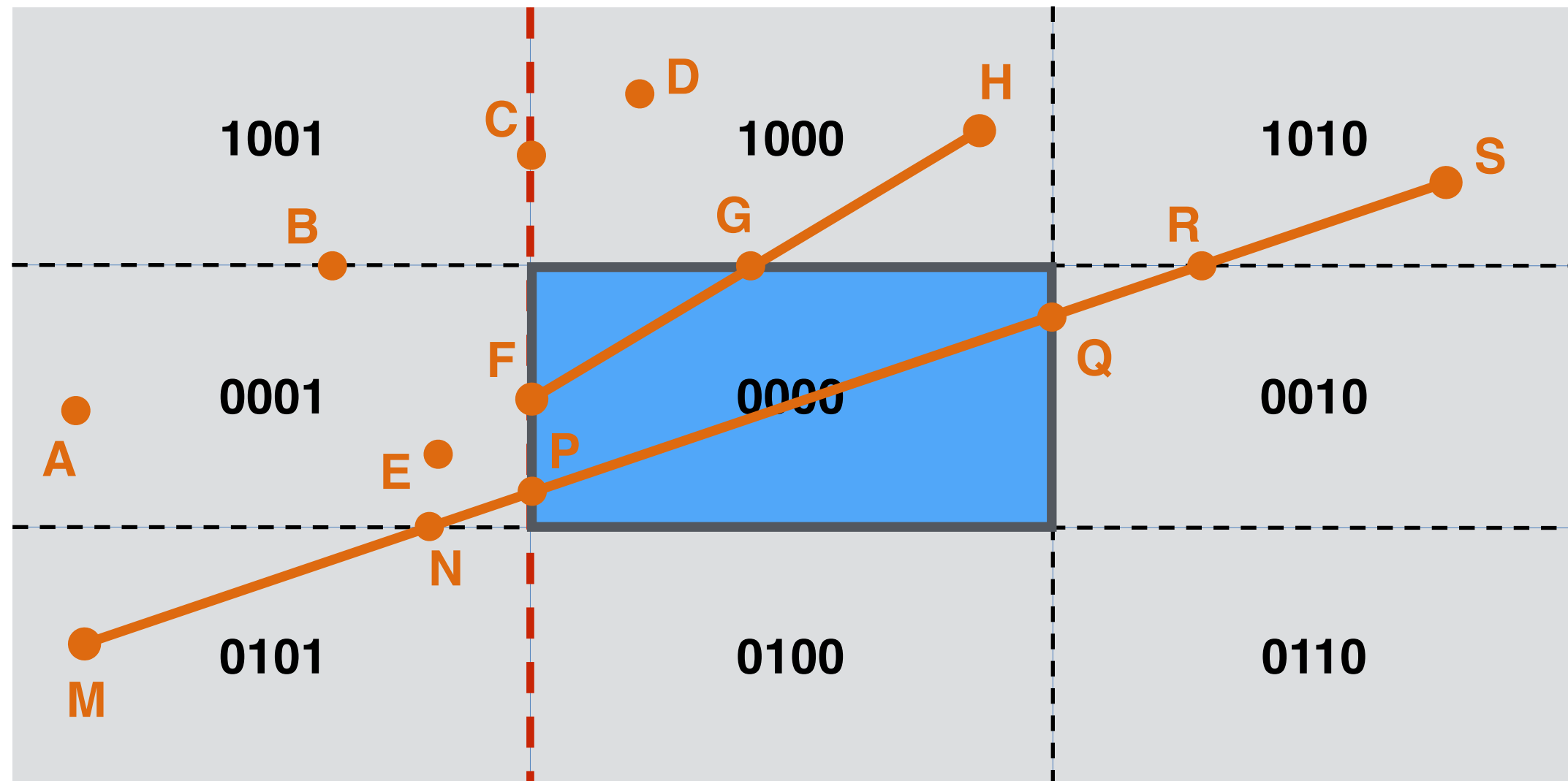
Algorithmul Cohen-Sutherland



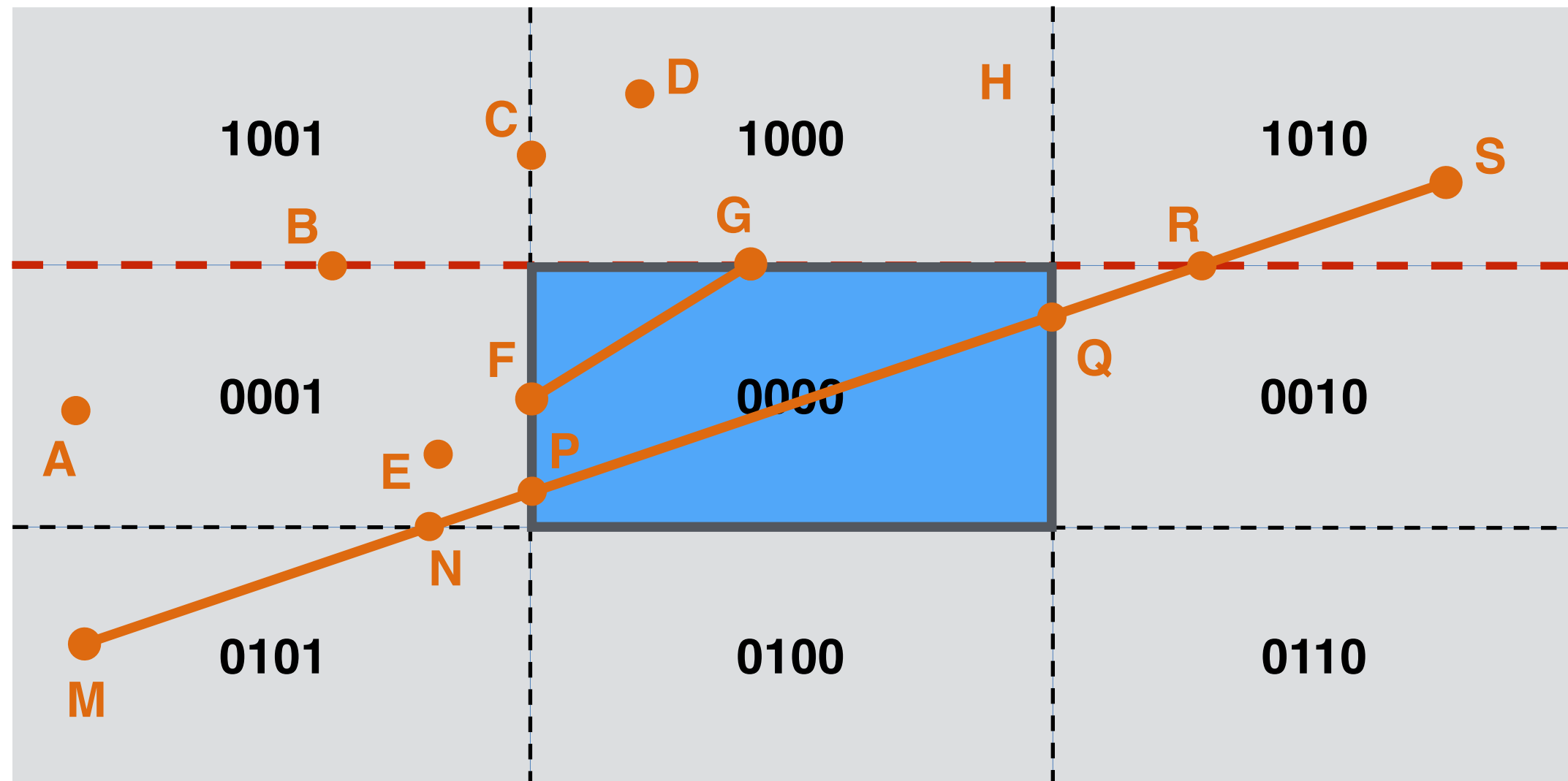
Algorithmul Cohen-Sutherland



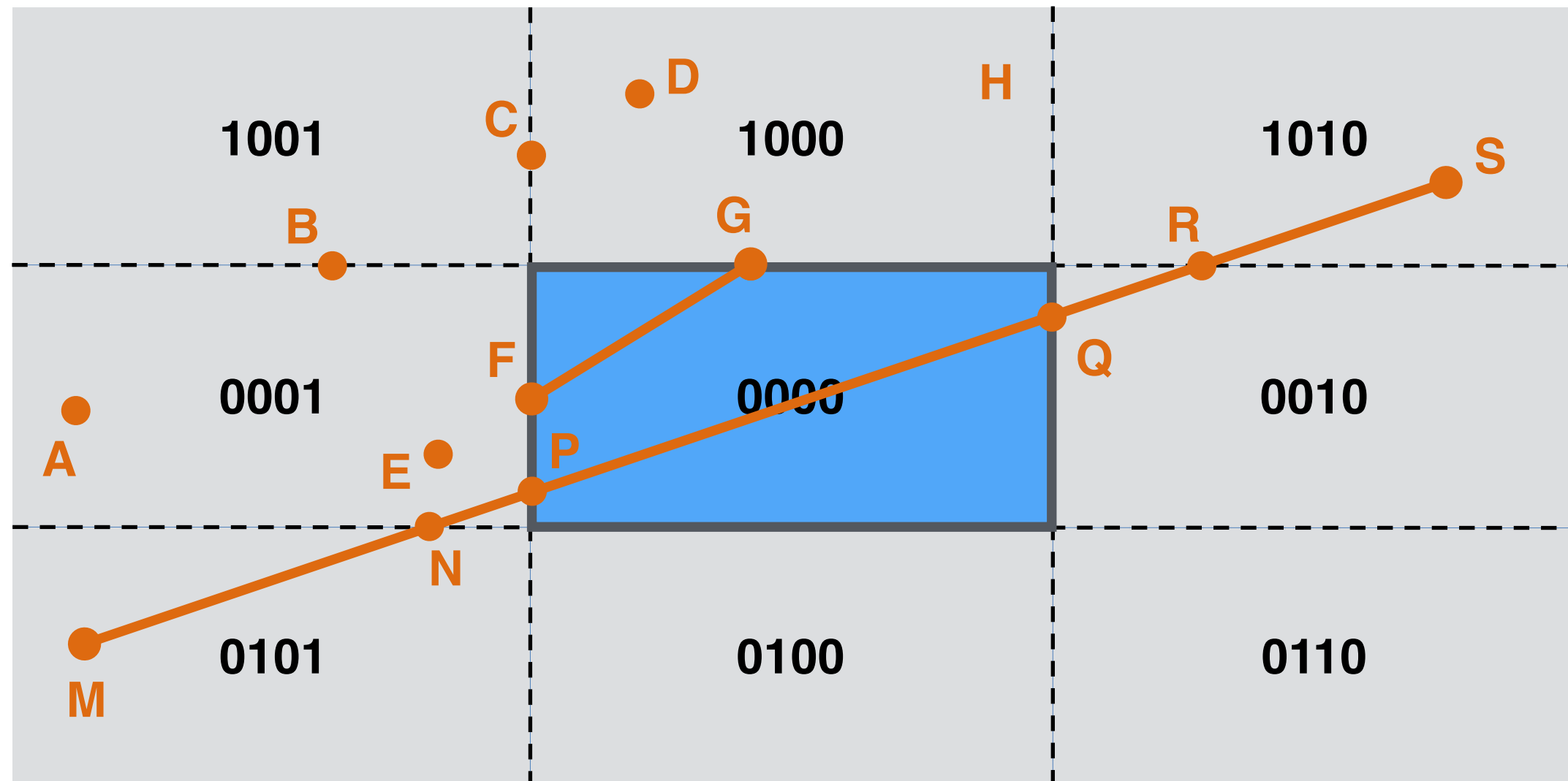
Algorithmul Cohen-Sutherland



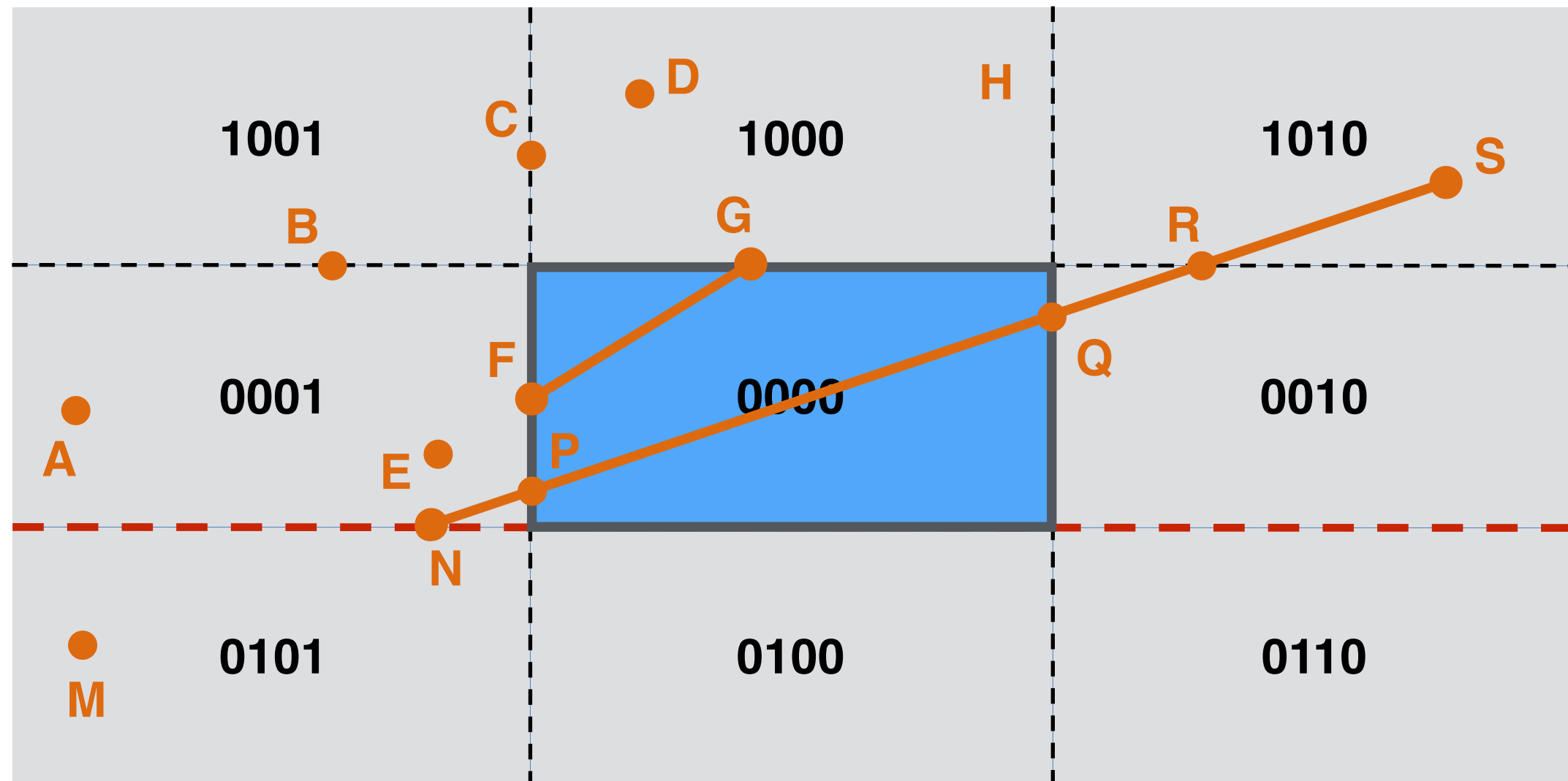
Algorithmul Cohen-Sutherland



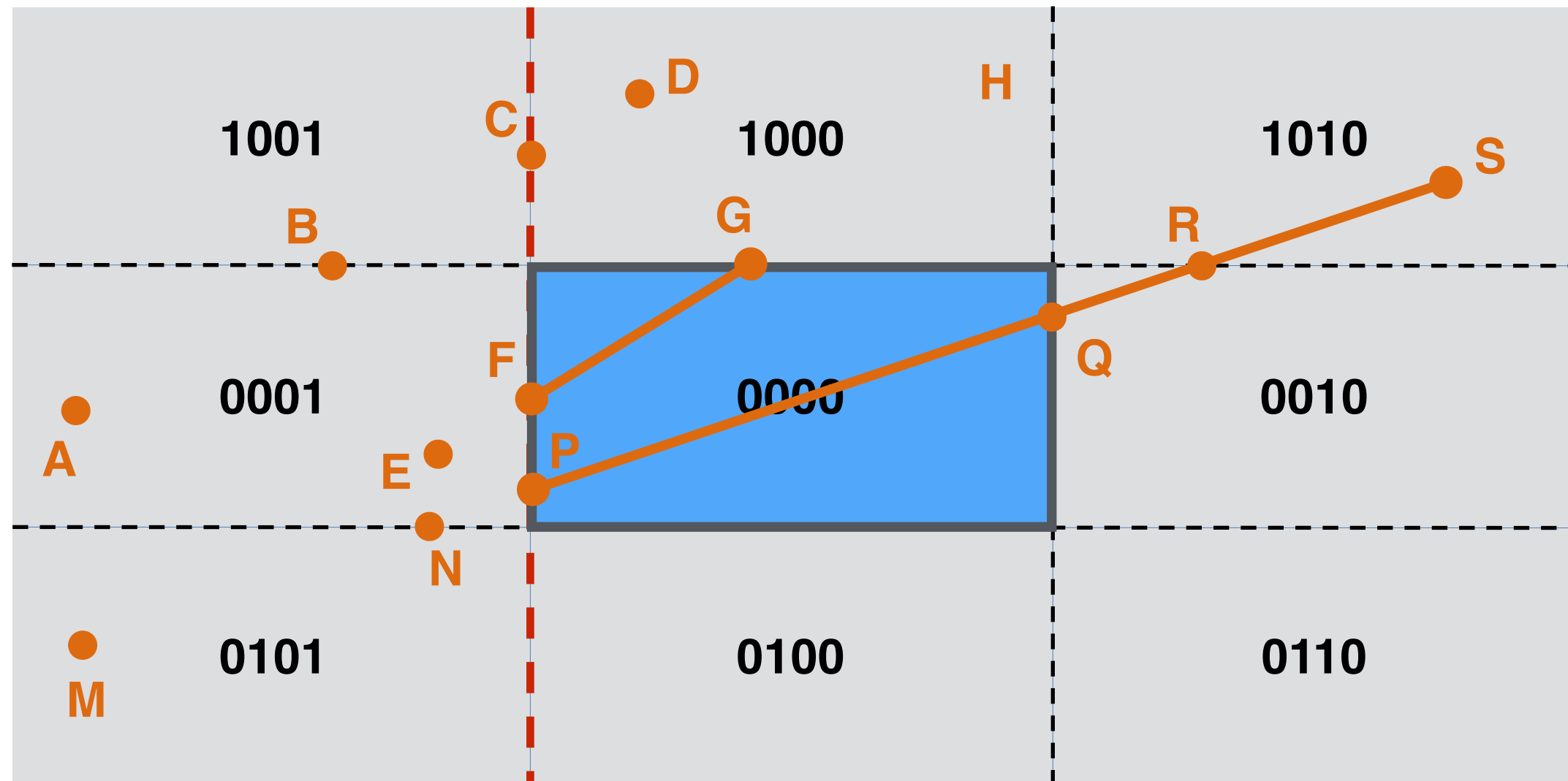
Algorithmul Cohen-Sutherland



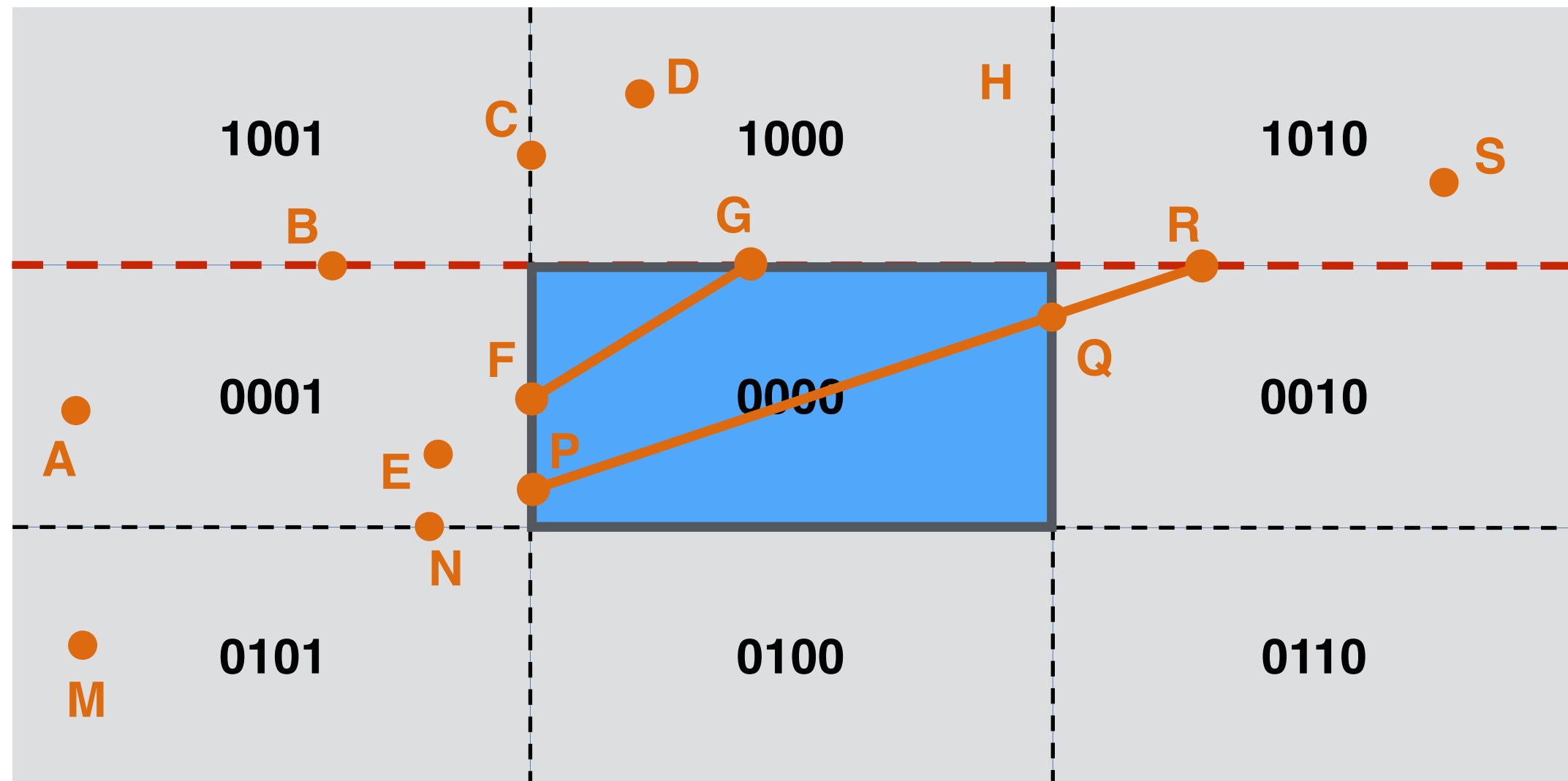
Algorithmul Cohen-Sutherland



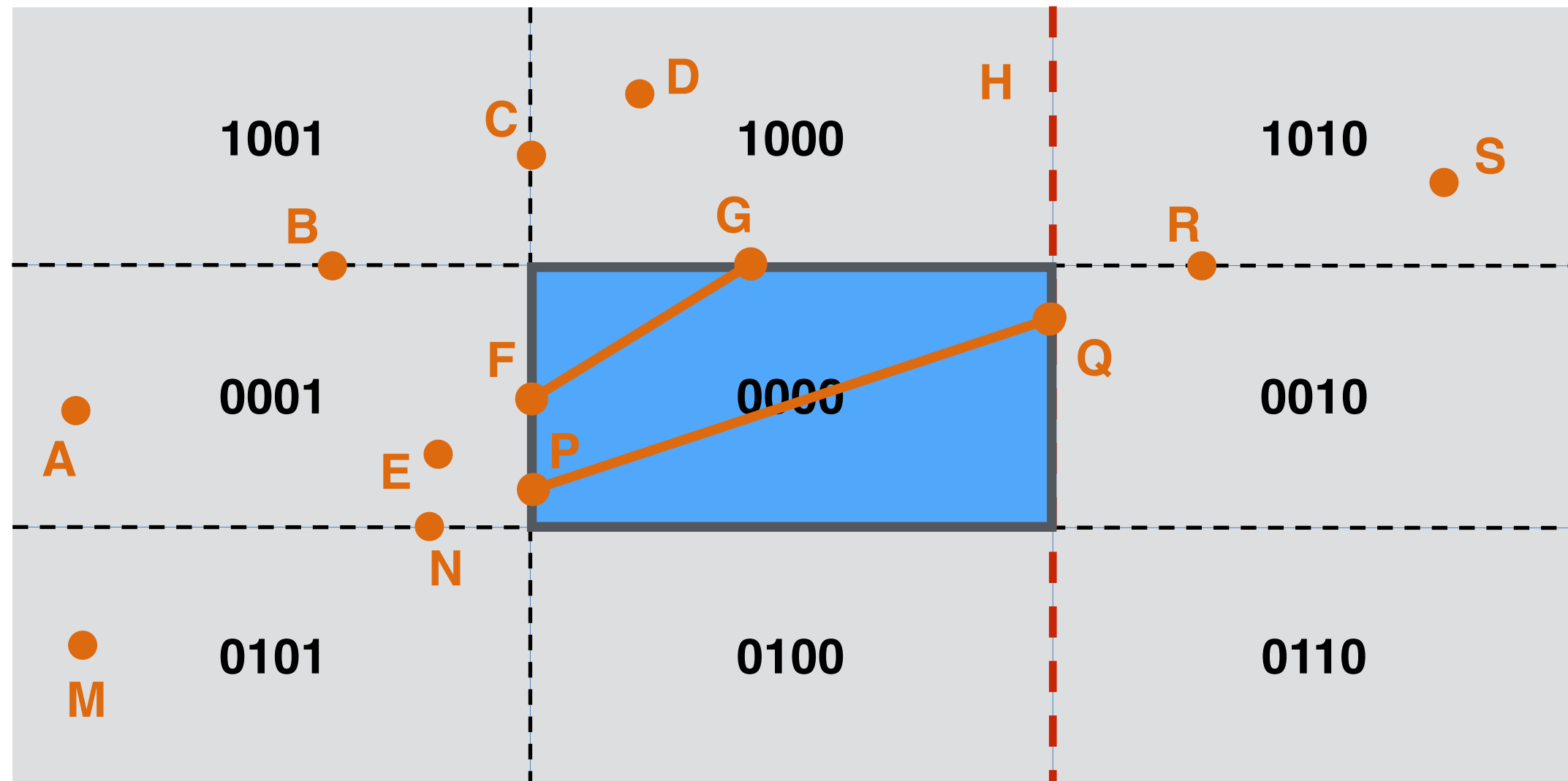
Algorithmul Cohen-Sutherland



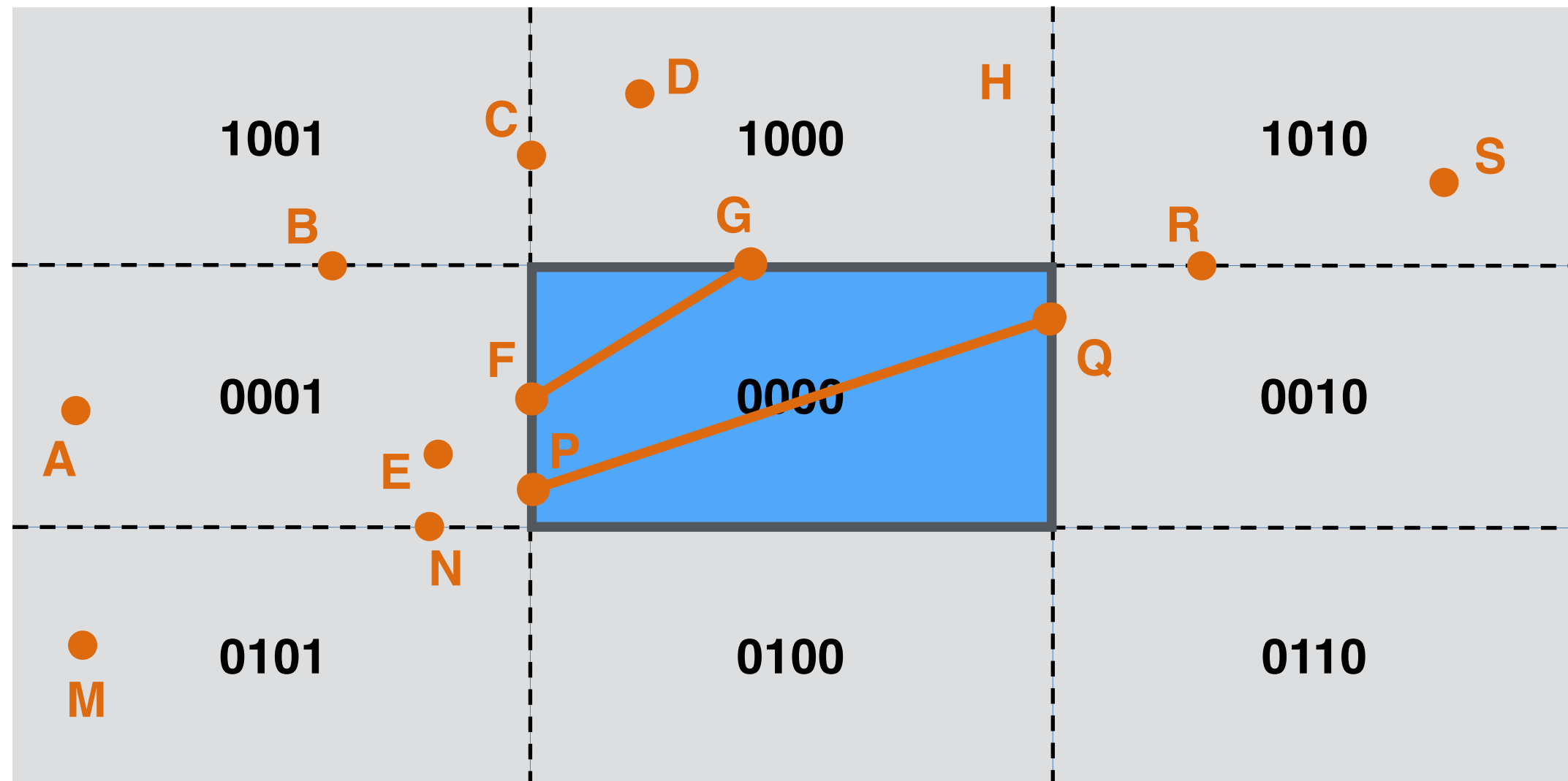
Algorithmul Cohen-Sutherland



Algorithmul Cohen-Sutherland



Algorithmul Cohen-Sutherland



Algoritmul Cohen-Sutherland

```
ComputeOutCode(x0, y0, outcode0);
```

```
ComputeOutCode(x1, y1, outcode1);
```

```
repeat{
```

```
    check for trivial reject or trivial accept;
```

```
    pick the point that is outside the clip rectangle;
```

```
    if(outcode & TOP){
```

```
         $x = x0 + (x1 - x0) * (ymax - y0) / (y1 - y0);$ 
```

```
         $y = ymax;$ 
```

```
    }else if(outcode & BOTTOM){
```

```
         $x = x0 + (x1 - x0) * (ymin - y0) / (y1 - y0);$ 
```

```
         $y = ymin;$ 
```

```
    }else if(outcode & RIGHT){
```

```
         $y = y0 + (y1 - y0) * (xmax - x0) / (x1 - x0);$ 
```

```
         $x = xmax;$ 
```

```
    }else if(outcode & LEFT){
```

```
         $y = y0 + (y1 - y0) * (xmin - x0) / (x1 - x0);$ 
```

```
         $x = xmin;$ 
```

```
    }
```

```
    if (x0, y0 is the outside point){
```

```
         $x0 = x; y0 = y; \text{ComputeOutCode}(x0, y0, \text{outcode0})$ 
```

```
    }else{
```

```
         $x1 = x; y1 = y; \text{ComputeOutCode}(x1, y1, \text{outcode1})$ 
```

```
    }
```

```
}until done
```

```
// divide line at top of clip rectangle
```

```
// divide line at bottom of clip rectangle
```

```
// divide line at right of clip rectangle
```

```
// divide line at left of clip rectangle
```


Algoritmul Cohen-Sutherland

ComputeOutCode(x0, y0, outcode0);

ComputeOutCode(x1, y1, outcode1);

repeat{

check for trivial reject or trivial accept;

pick the point that is outside the clip rectangle;

if(outcode & TOP){

$x = x_0 + (x_1 - x_0) * (y_{max} - y_0) / (y_1 - y_0);$

$y = y_{max};$

}else if(outcode & BOTTOM){

$x = x_0 + (x_1 - x_0) * (y_{min} - y_0) / (y_1 - y_0);$

$y = y_{min};$

}else if(outcode & RIGHT){

$y = y_0 + (y_1 - y_0) * (x_{max} - x_0) / (x_1 - x_0);$

$x = x_{max};$

}else if(outcode & LEFT){

$y = y_0 + (y_1 - y_0) * (x_{min} - x_0) / (x_1 - x_0);$

$x = x_{min};$

}

if (x0, y0 is the outside point){

$x_0 = x; y_0 = y; \text{ComputeOutCode}(x_0, y_0, \text{outcode}_0)$

}else{

$x_1 = x; y_1 = y; \text{ComputeOutCode}(x_1, y_1, \text{outcode}_1)$

}

}until done

$$P(t) = P_0 + (P_1 - P_0)t$$

$$x' = x_0 + (x_1 - x_0)t$$

$$y' = y_0 + (y_1 - y_0)t = y_{max}$$

$$t = \frac{y_{max} - y_0}{y_1 - y_0}$$

$$x' = x_0 + (x_1 - x_0) \frac{y_{max} - y_0}{y_1 - y_0}$$

Decuparea unei linii definite parametric

Idea de bază:

1. Definirea liniei de decupat și a muchiilor ferestrei de decupare parametric

e.g. linia definită de $(\mathbf{x}_0, \mathbf{y}_0)$ și unde $(\mathbf{x}_1, \mathbf{y}_1)$:

$$\mathbf{P} = \mathbf{P}_0 + \mathbf{t}_{\text{line}} (\mathbf{P}_1 - \mathbf{P}_0), \text{ where } \mathbf{t}_{\text{line}} \in [0, 1]$$

2. Rezolvarea sistemului:

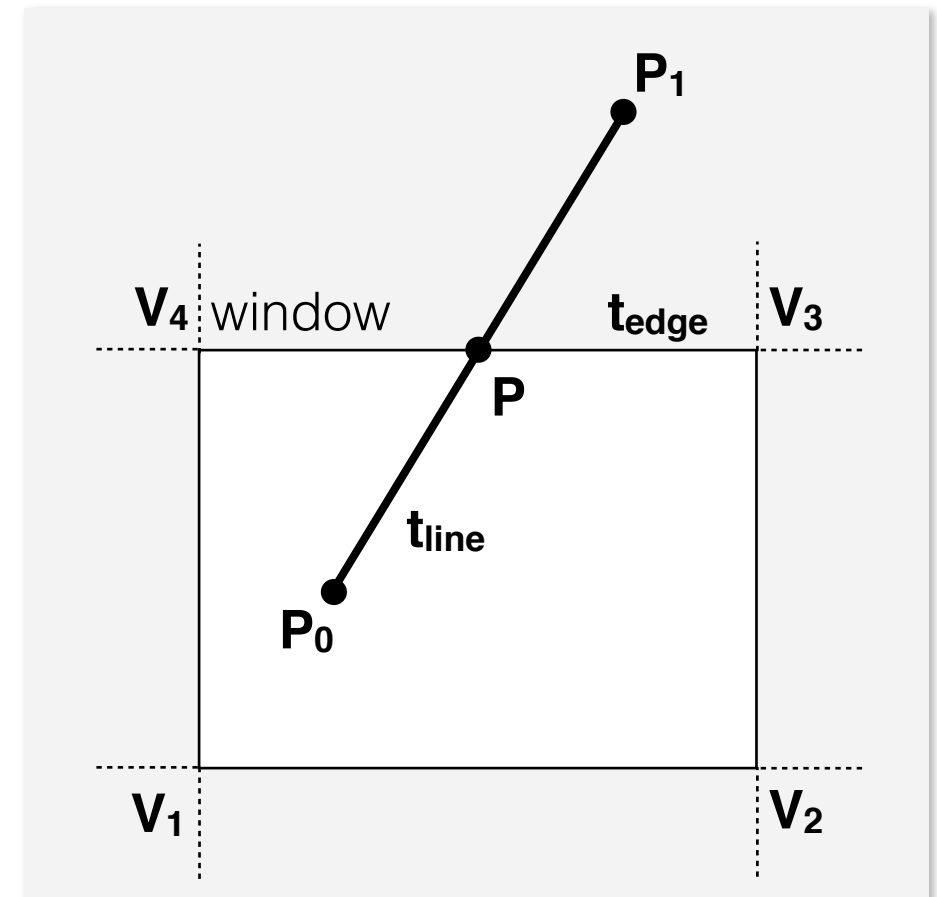
$$\mathbf{P} = \mathbf{P}_0 + \mathbf{t}_{\text{line}} (\mathbf{P}_1 - \mathbf{P}_0)$$

$$\mathbf{P} = \mathbf{V}_i + \mathbf{t}_{\text{edge}} (\mathbf{V}_{i+1} - \mathbf{V}_i)$$

3. Linia și muchia (muchiiile) se intersectează

dacă $\mathbf{t}_{\text{edge}}$ și $\mathbf{t}_{\text{line}} \in [0, 1]$

4. Linia decupată este $\mathbf{P}_0\mathbf{P}(\mathbf{t})$, $\mathbf{t}_{\text{line}} \in [0, 1]$



Algoritmi de decupare:

- 1978 Cyrus-Beck algorithm
- 1984 Liang-Barsky algorithm (improved form of the Cyrus-Beck algorithm)

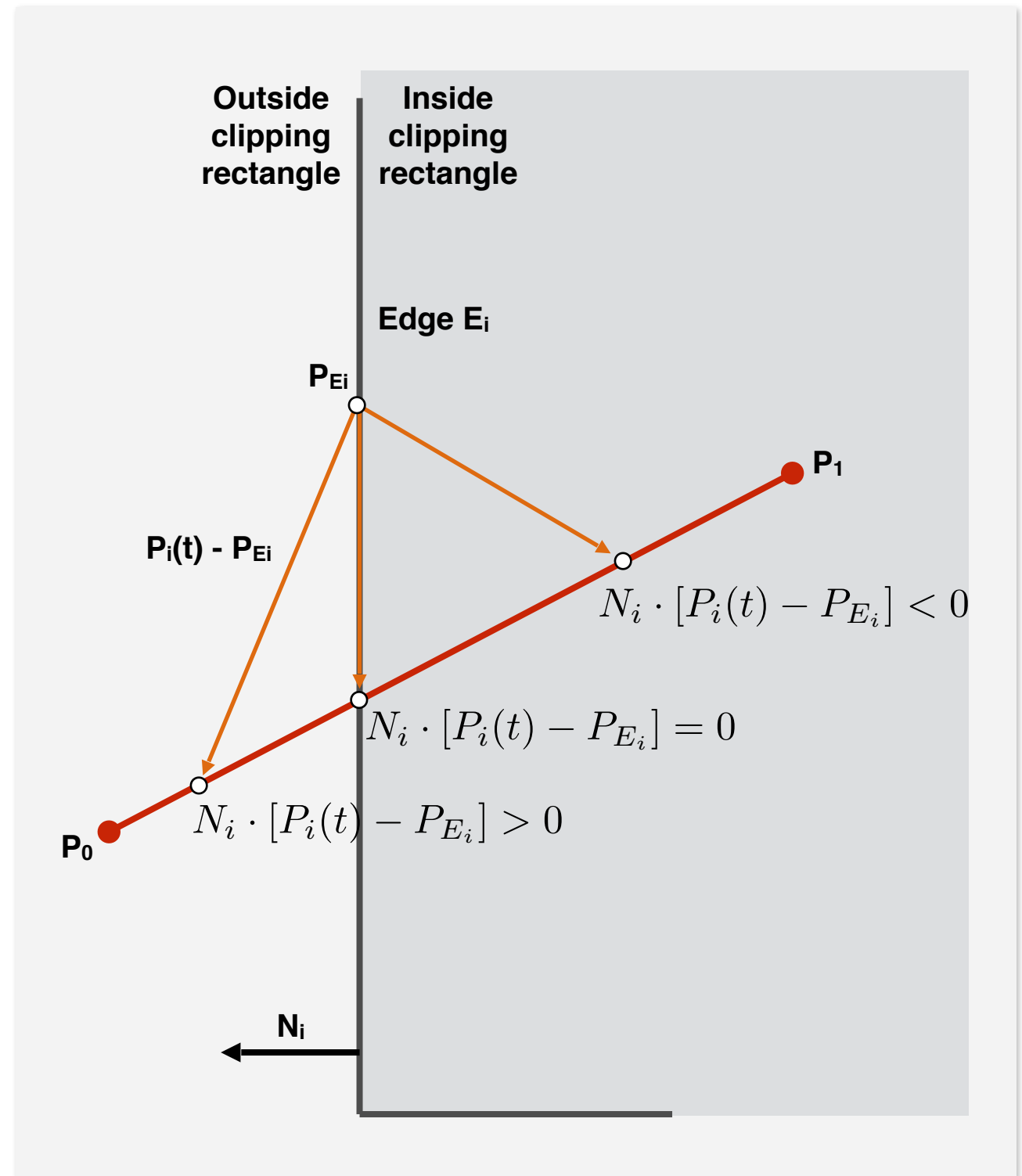
Algorithmul Cyrus-Beck

$$N_i \cdot [P_i(t) - P_{E_i}] = \|N_i\| \|P_i(t) - P_{E_i}\| \cos \theta$$

$$N_i \cdot [P_i(t) - P_{E_i}] < 0 \quad \longrightarrow \quad P \text{ inside}$$

$$N_i \cdot [P_i(t) - P_{E_i}] = 0 \quad \longrightarrow \quad P \text{ on the edge}$$

$$N_i \cdot [P_i(t) - P_{E_i}] > 0 \quad \longrightarrow \quad P \text{ outside}$$



Algoritmul Cyrus-Beck

Rezolvăm ecuația:

$$N_i \cdot [P_i(t) - P_{E_i}] = 0$$

Înlocuim: $P(t) = P_0 + (P_1 - P_0)t$

$$N_i \cdot [P_0 + (P_1 - P_0)t - P_{E_i}] = 0$$

$$N_i \cdot [P_0 - P_{E_i}] + N_i \cdot [P_1 - P_0]t = 0$$

$D = (P_1 - P_0)$ este vectorul orientat de la P_0 la P_1

$$t = \frac{N_i \cdot [P_0 - P_{E_i}]}{-N_i \cdot D}$$

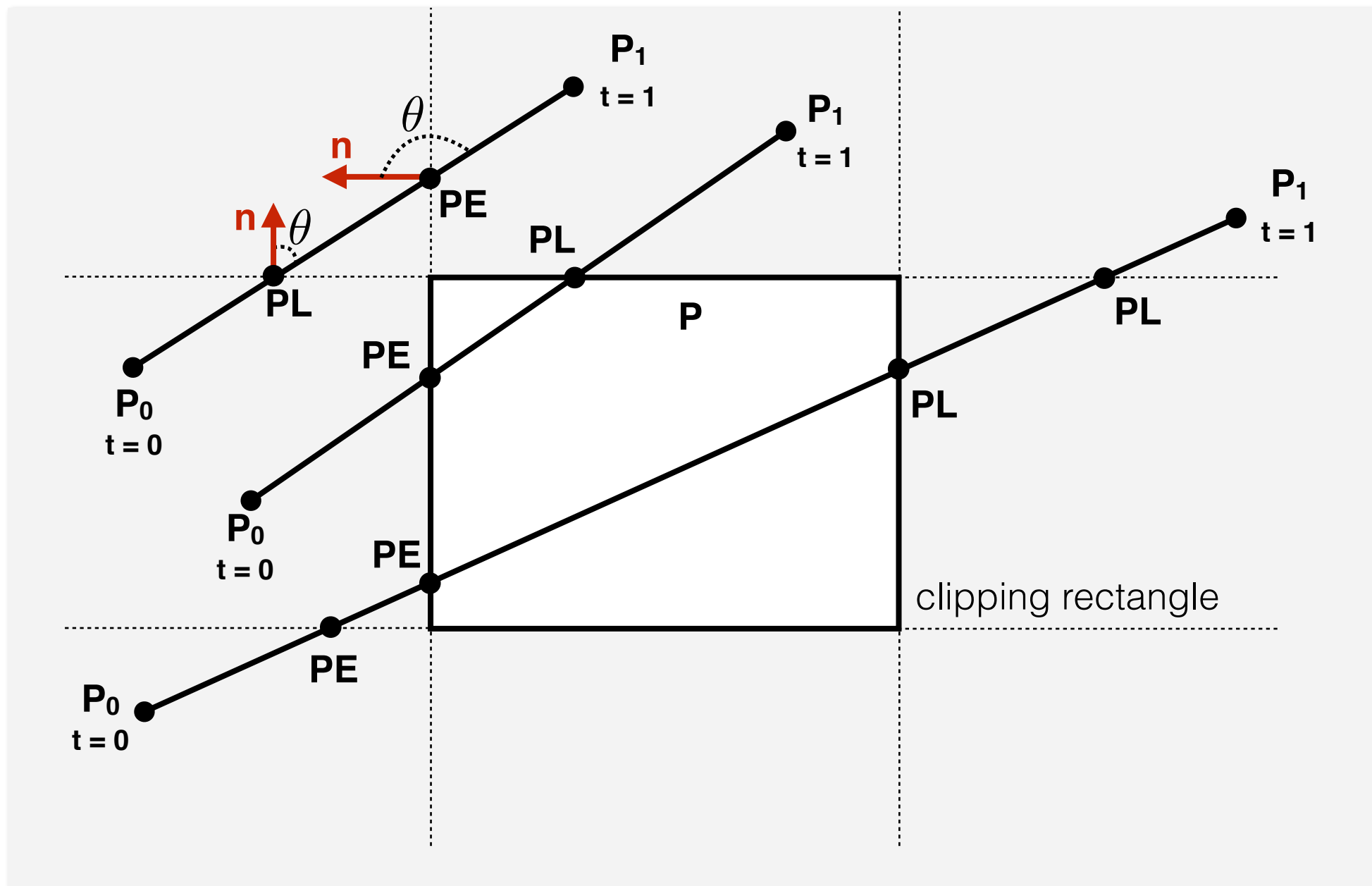
Algoritmul verifică:

$$N_i \neq 0$$

$$D \neq 0$$

$$N_i \cdot D \neq 0$$

Cyrus-Beck algorithm



PE = entering point: $N_i \cdot D < 0, \cos \theta < 0, \theta > 90^\circ$

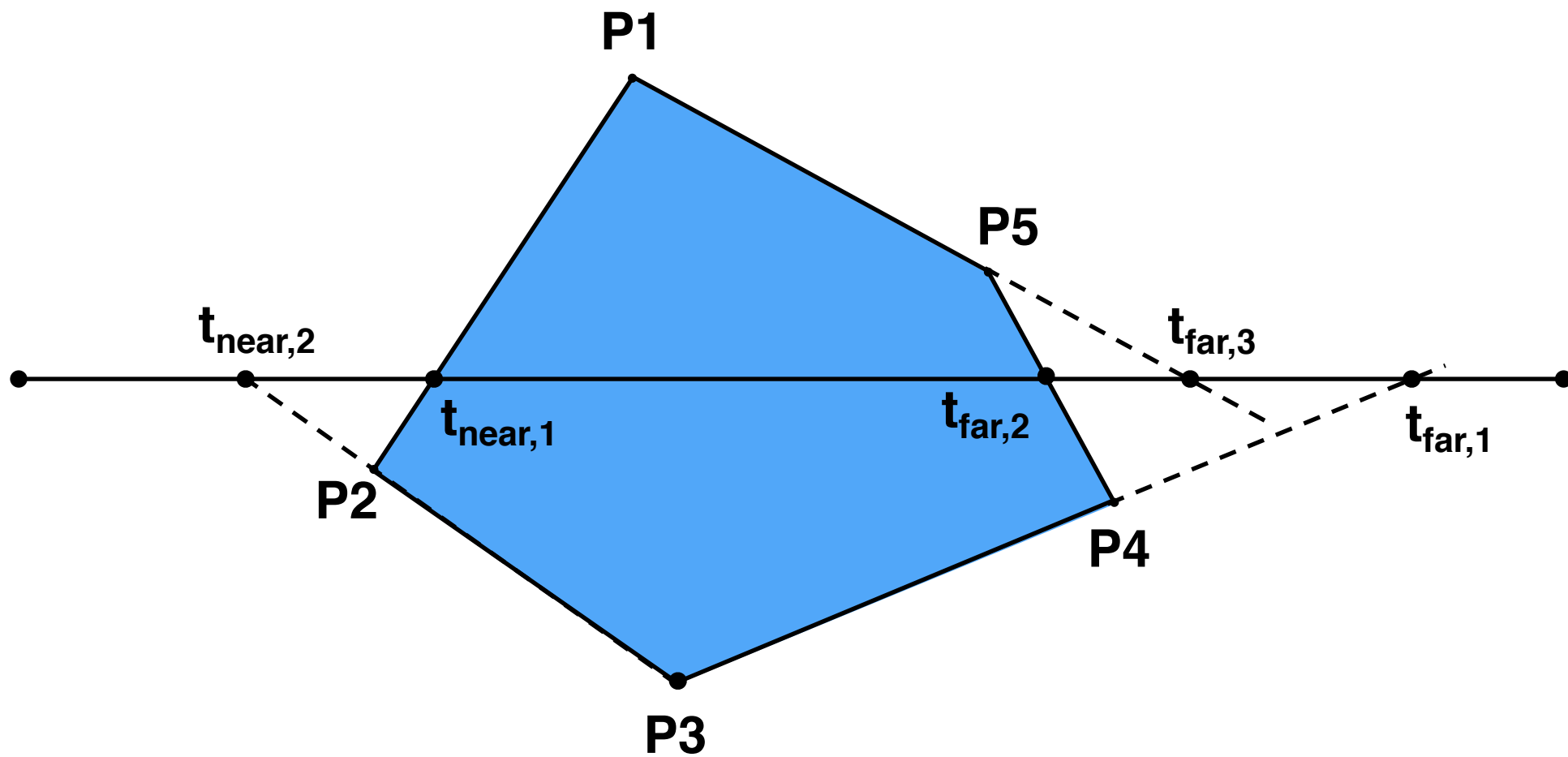
PL = leaving point: $N_i \cdot D > 0, \cos \theta > 0, \theta < 90^\circ$

$$D = (P_1 - P_0)$$

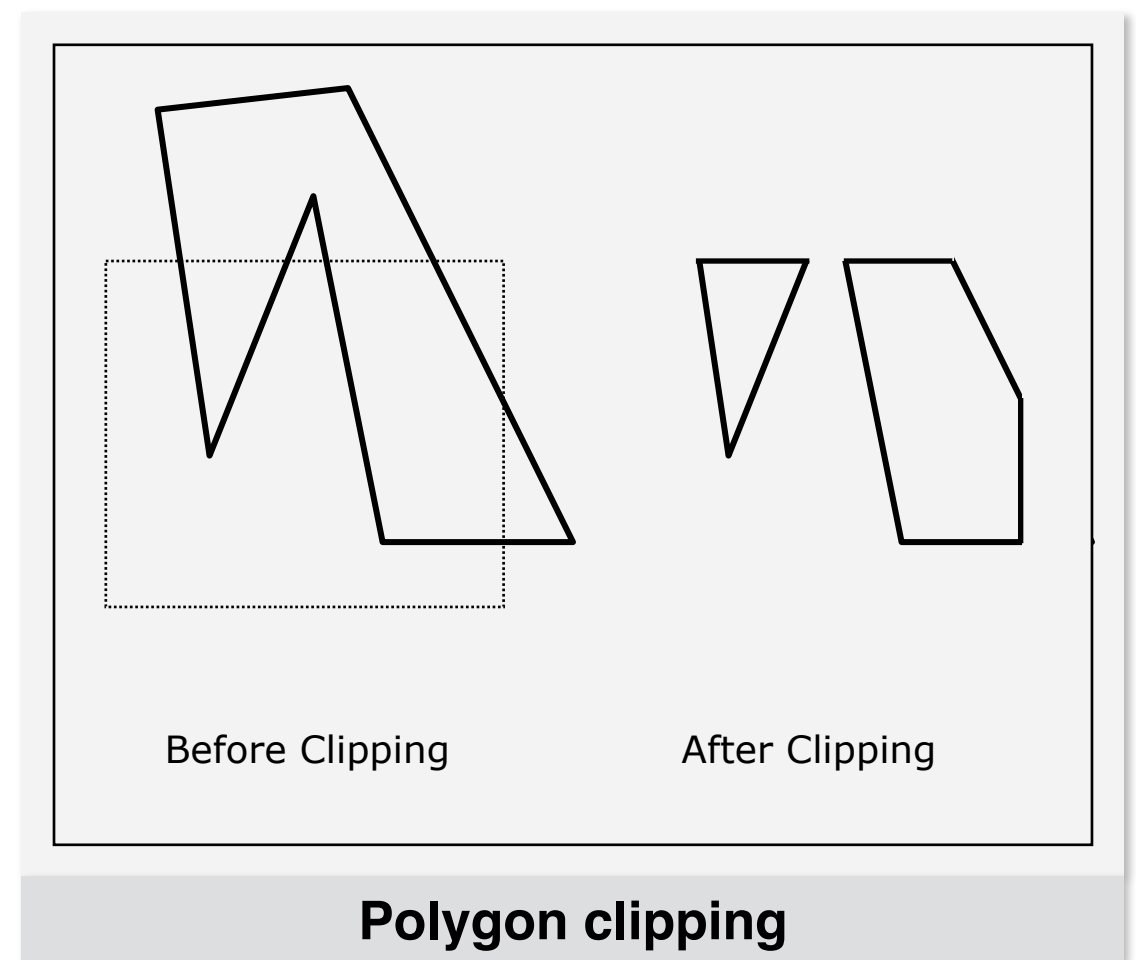
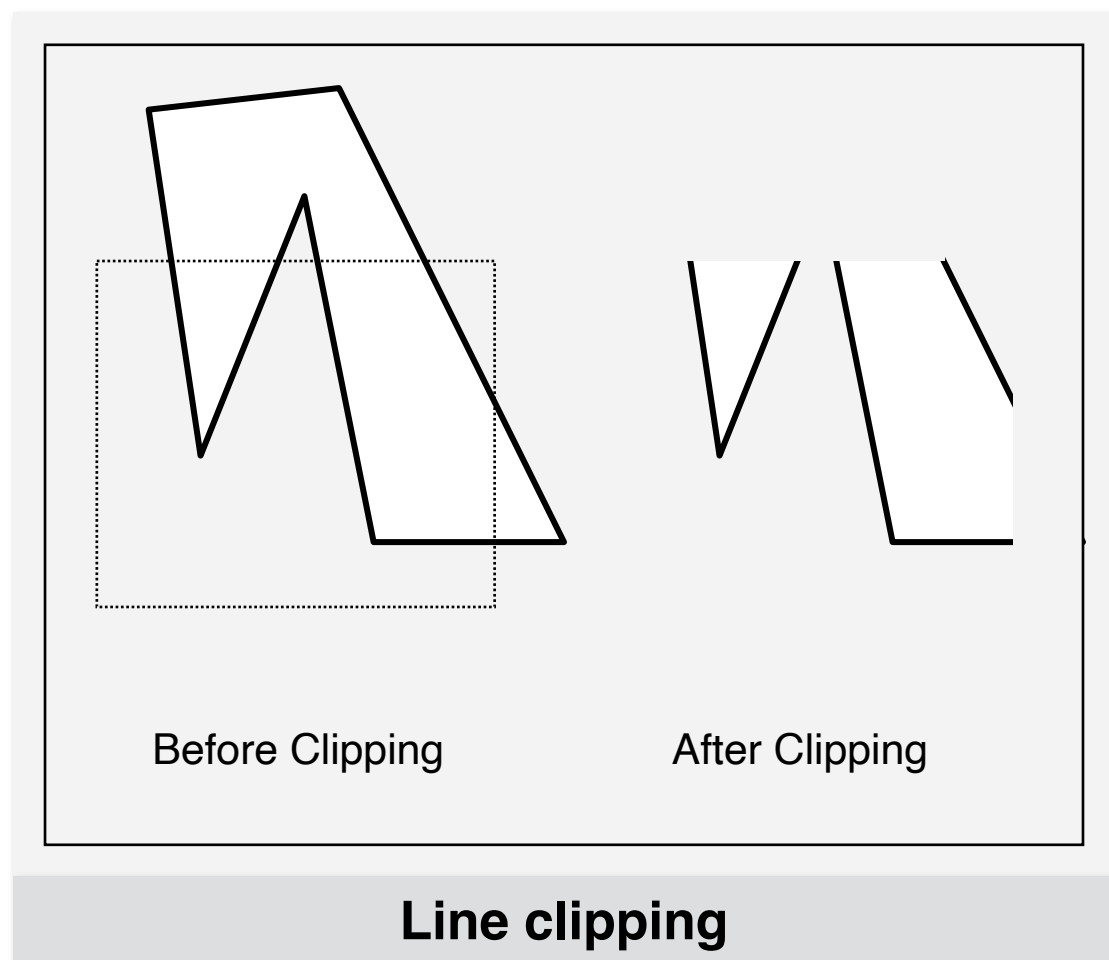
Cyrus-Beck algorithm - pseudocode

```
precalculate  $N_i$  and select a  $PE_i$  for each edge;
for (each line segment to be clipped){
    if ( $P_1 = P_0$ )
        line degenerates to a point, so clip as a point;
    else
         $t_E = 0; t_L = 1;$ 
        for (each candidate compute intersection with a clipping edge){
            if ( $N_i * D \neq 0$ ){ //ignore edges parallel to line
                calculate  $t$ ;
                use sign of  $N_i * D$  to categorize as PE (potentially entering) or PL (potentially leaving);
                if (PE)
                     $t_E = \max(t_E, t);$ 
                if (PL)
                     $t_L = \min(t_L, t);$ 
            }
        }
        if ( $t_E > t_L$ )
            return NULL
        else
            return  $P(t_E)$  and  $P(t_L)$  as true clip intersections
    }
```

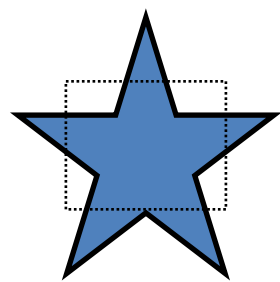
Cyrus-Beck algorithm



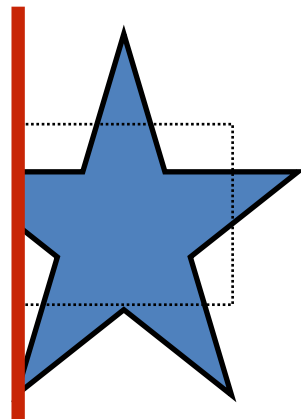
Decupareaa poligoanelor



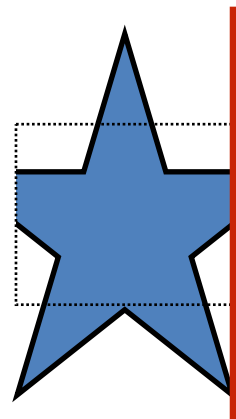
Algoritmul Sutherland-Hodgman



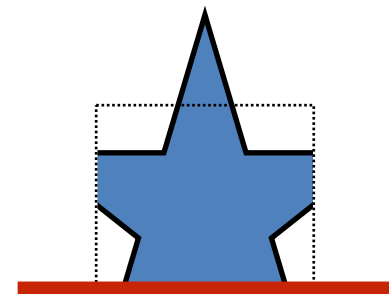
Original
Polygon



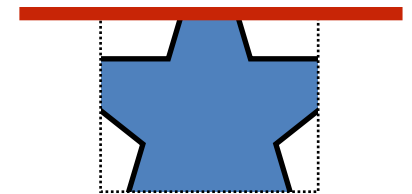
Clip
Left



Clip
Right

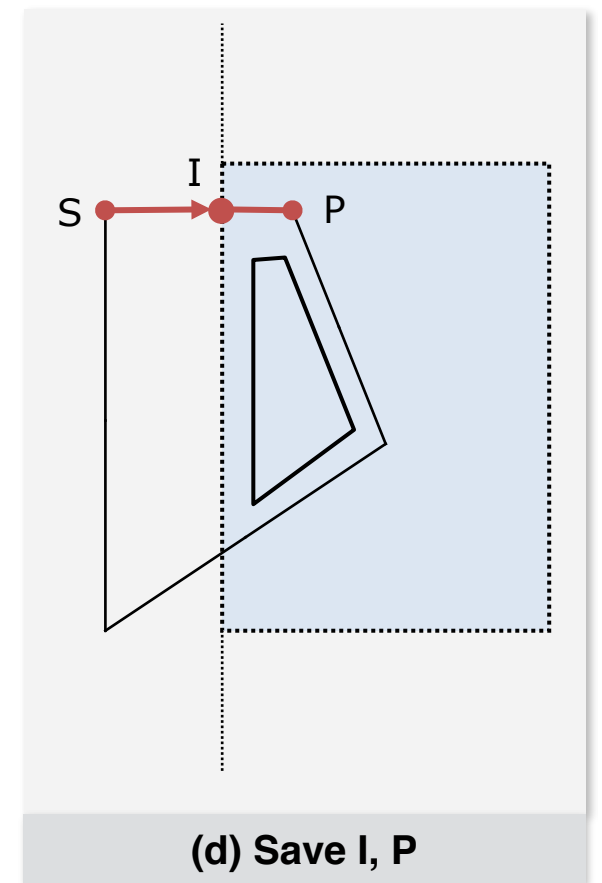
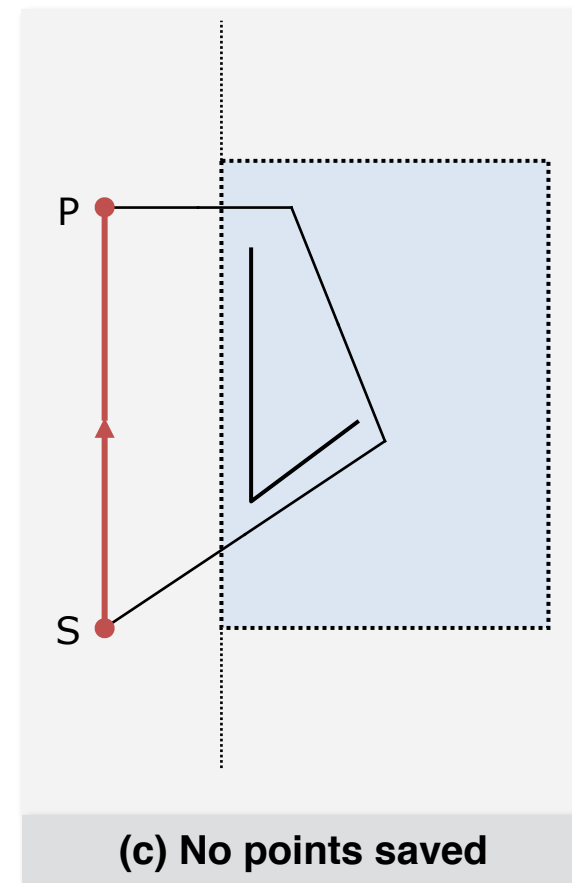
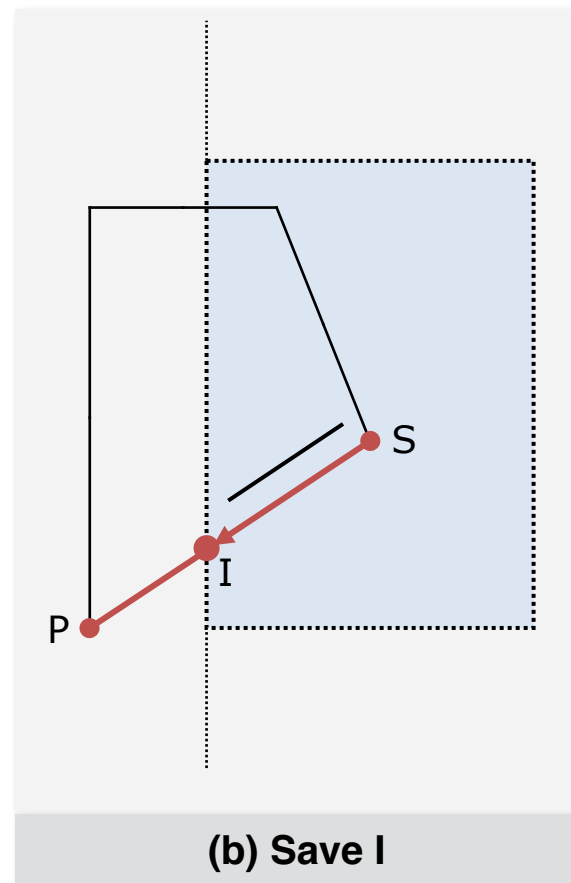
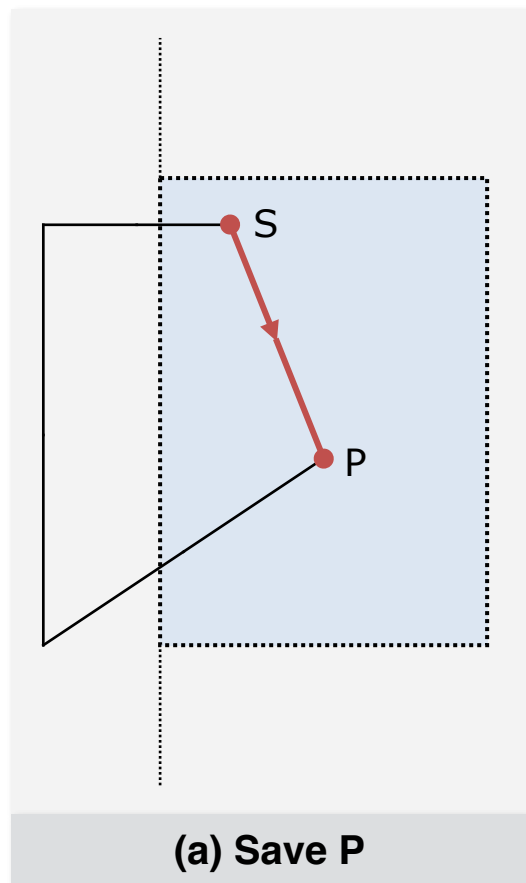


Clip
Bottom



Clip
Top

Algorithm Sutherland-Hodgman



Algoritmul Sutherland-Hodgman

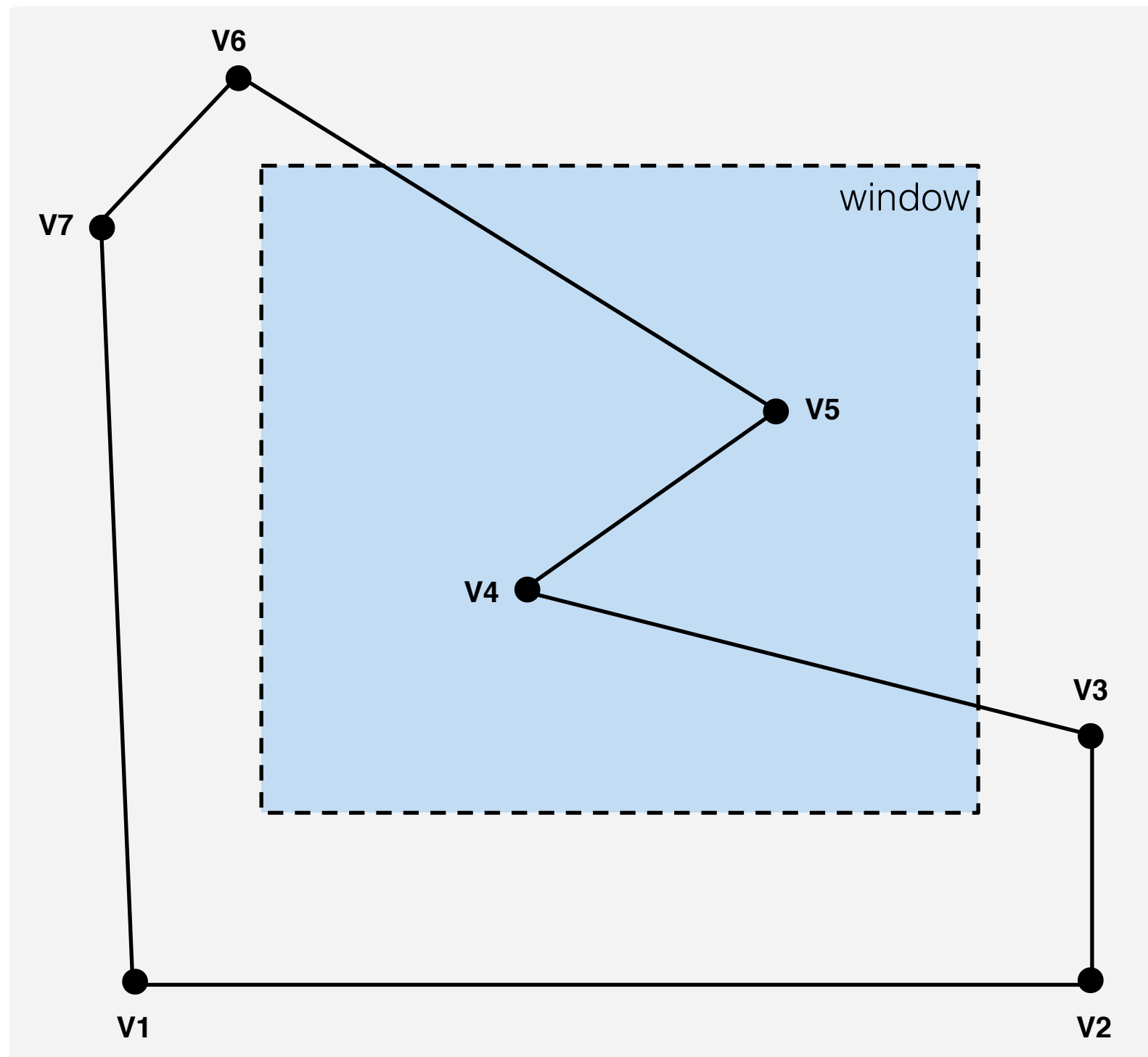
Input vertex list: $inv[] = \{v_1, v_2, \dots, v_n\}$
Output vertex list: $outv[] = \{v_1', v_2', \dots, v_p'\}$

```
ClippingSH(vertexList inputVertices, vertexList outputVertices, windowEdge clippingEdges[4])  
{  
    for(each clipping edge){  
        EdgeClippingSH(inputVertices, outputVertices, clippingEdges[currentEdge]);  
        inputVertices = outputVertices; // update the current edge list  
    }  
}
```

Algoritmul Sutherland-Hodgman

```
EdgeClippingSH(vertexList inputVertices, vertexList outputVertices, windowEdge  
clippingEdge)  
{  
    vertex i,p,s;  
    select s to be the last vertex in inputVertices;  
  
    for(each vertex p from inputVertices){  
        if(ToWindow(p, clippingEdge))           //case a and d  
            if(ToWindow(s, clippingEdge))       //case a  
                write vertex p into outputVertices;  
            else{  
                i = Intersection(s, p, clippingEdge);  
                write intersection i into outputVertices;  
                write vertex p into outputVertices;  
            }  
            else if(ToWindow(s, clippingEdge)){ //case b  
                i = Intersection(s, p, clippingEdge);  
                write intersection i into outputVertices;  
            }                               //for case c don't write anything  
            //update the start vertex  
            s = p;  
        }  
    }  
}
```

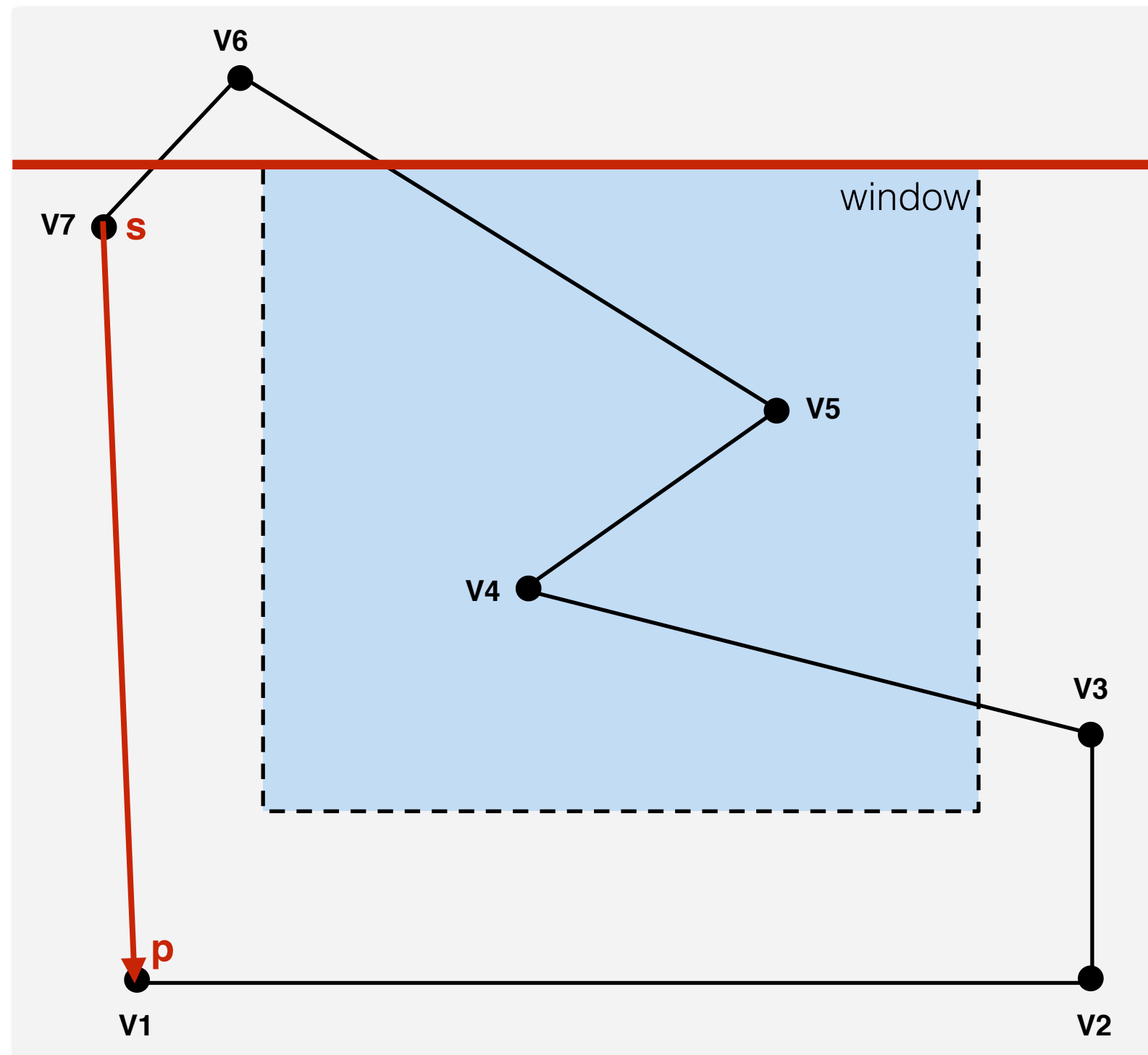
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V6, V7

outputVertices:

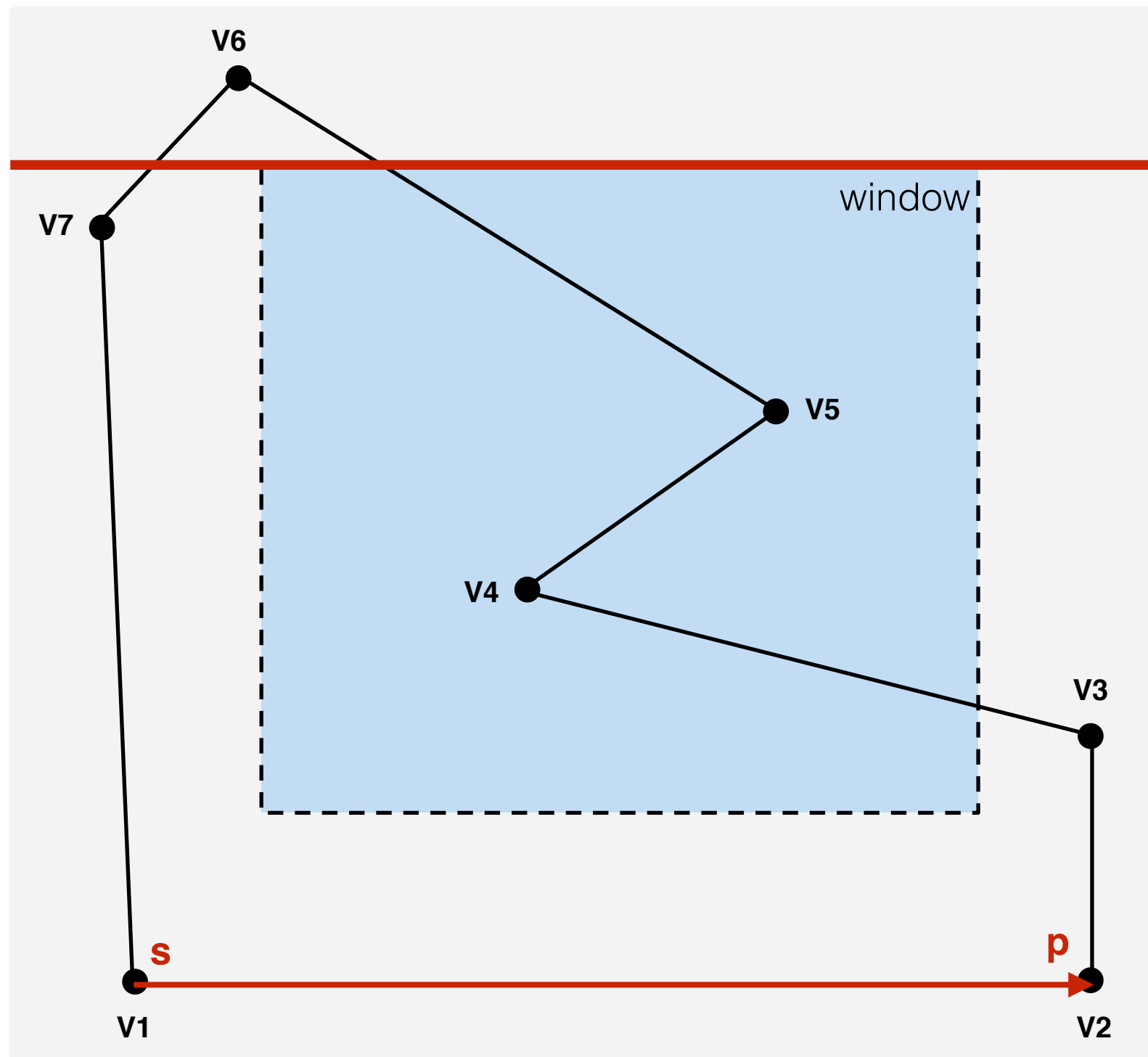
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V6, V7

outputVertices: V1

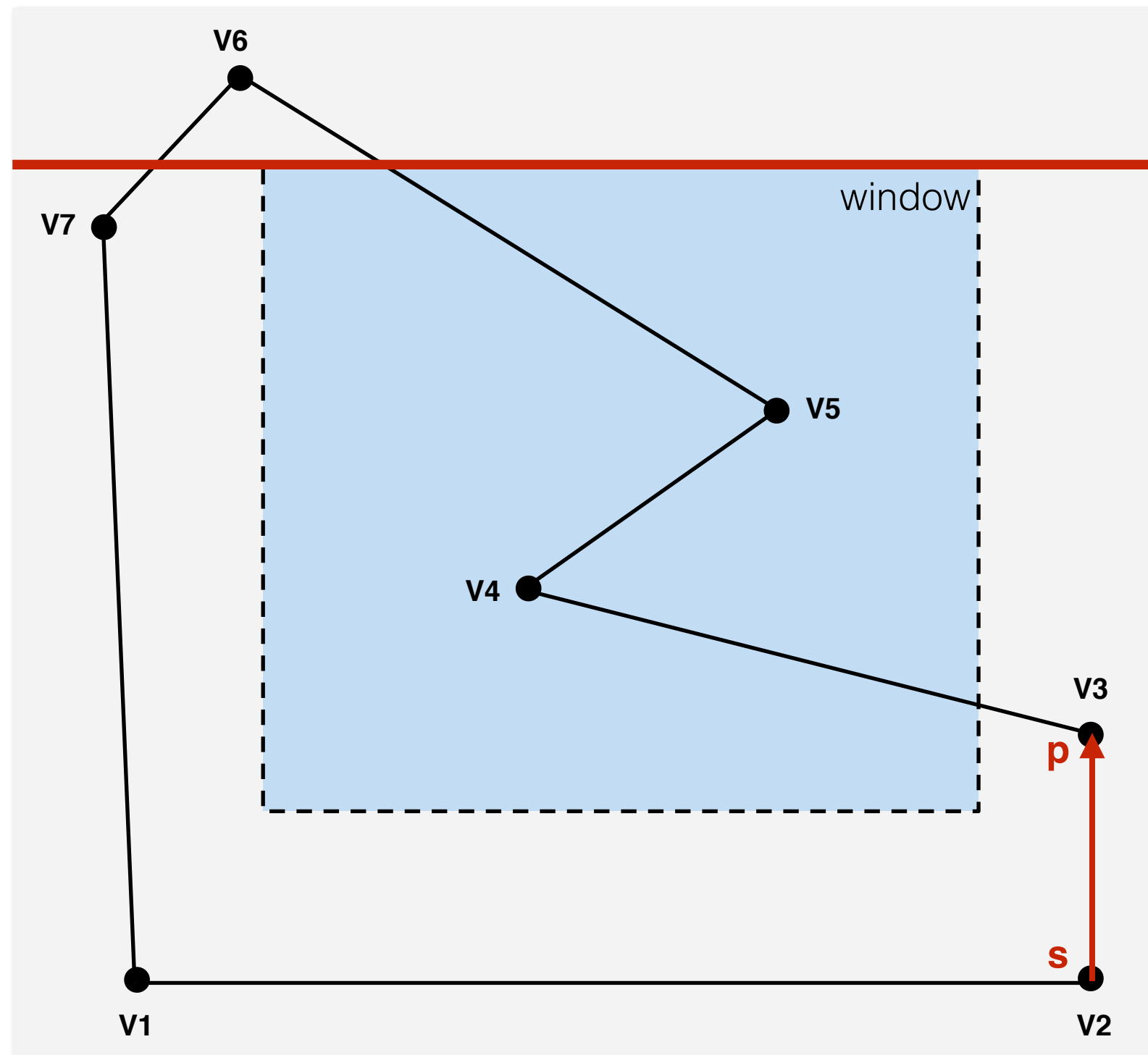
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V6, V7

outputVertices: V1, V2

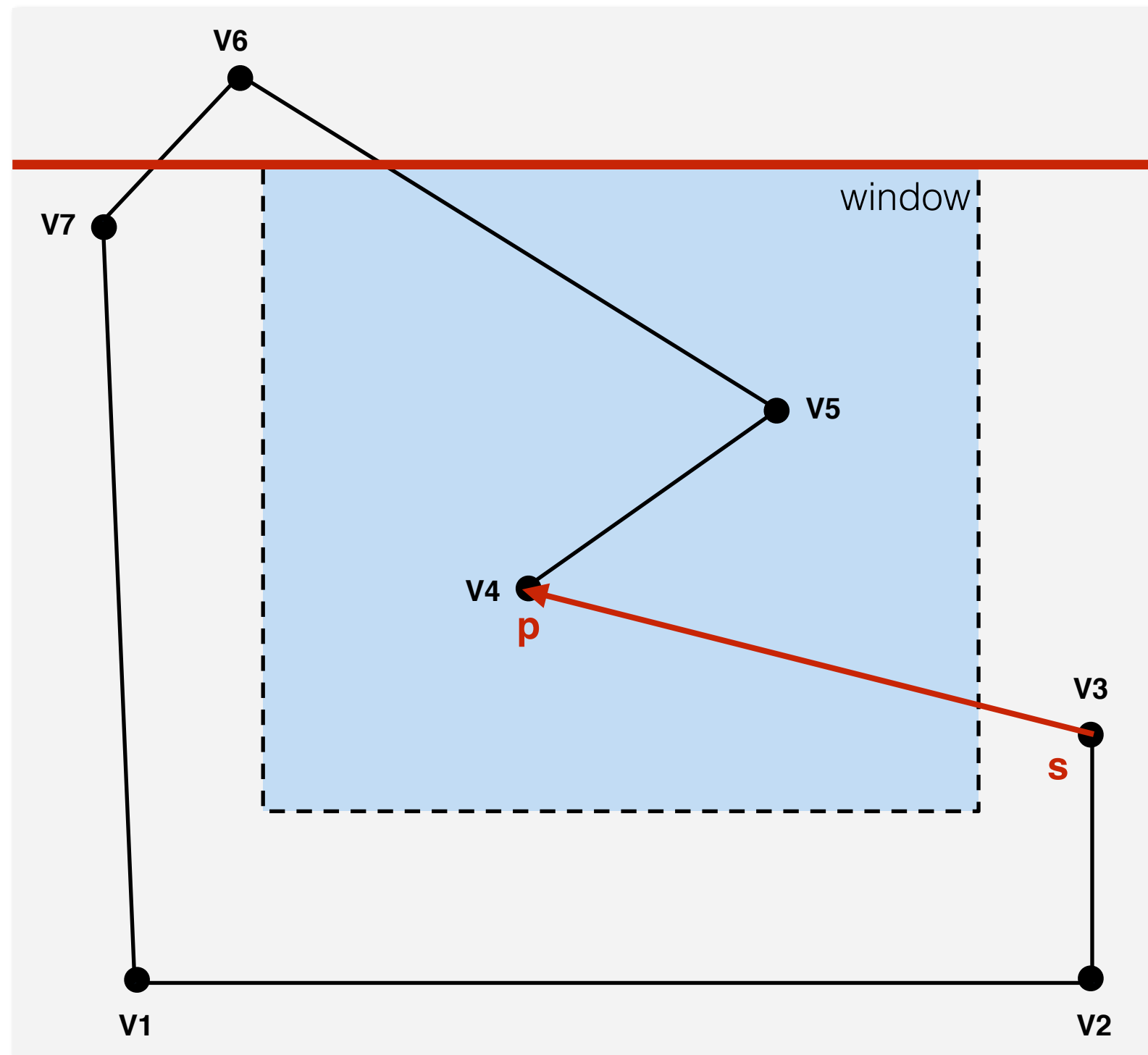
Algorithm Sutherland-Hodgman



inputVertices: $V1, V2, V3, V4, V5, V6, V7$

outputVertices: $V1, V2, V3$

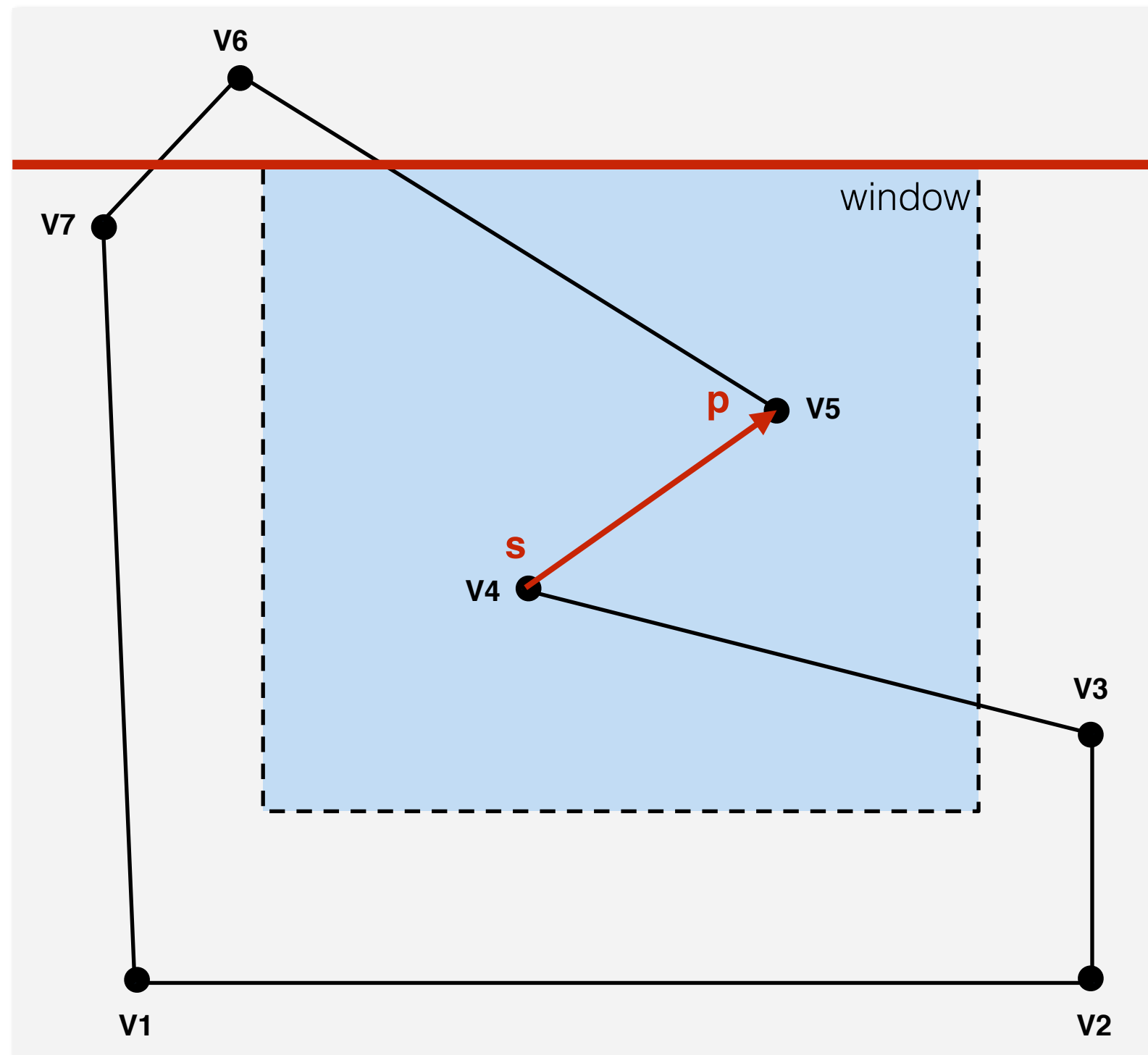
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V6, V7

outputVertices: V1, V2, V3, V4

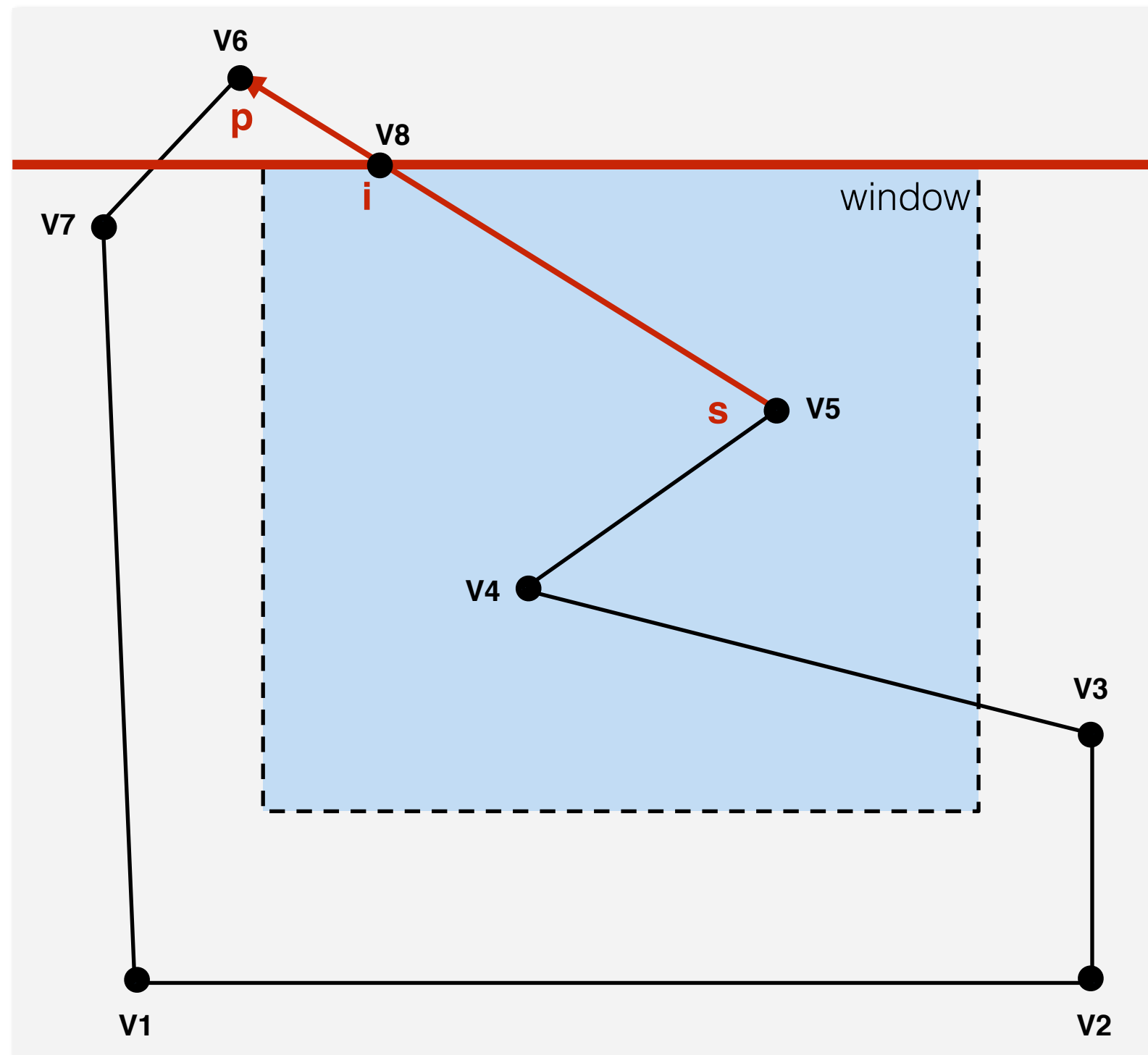
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V6, V7

outputVertices: V1, V2, V3, V4, V5

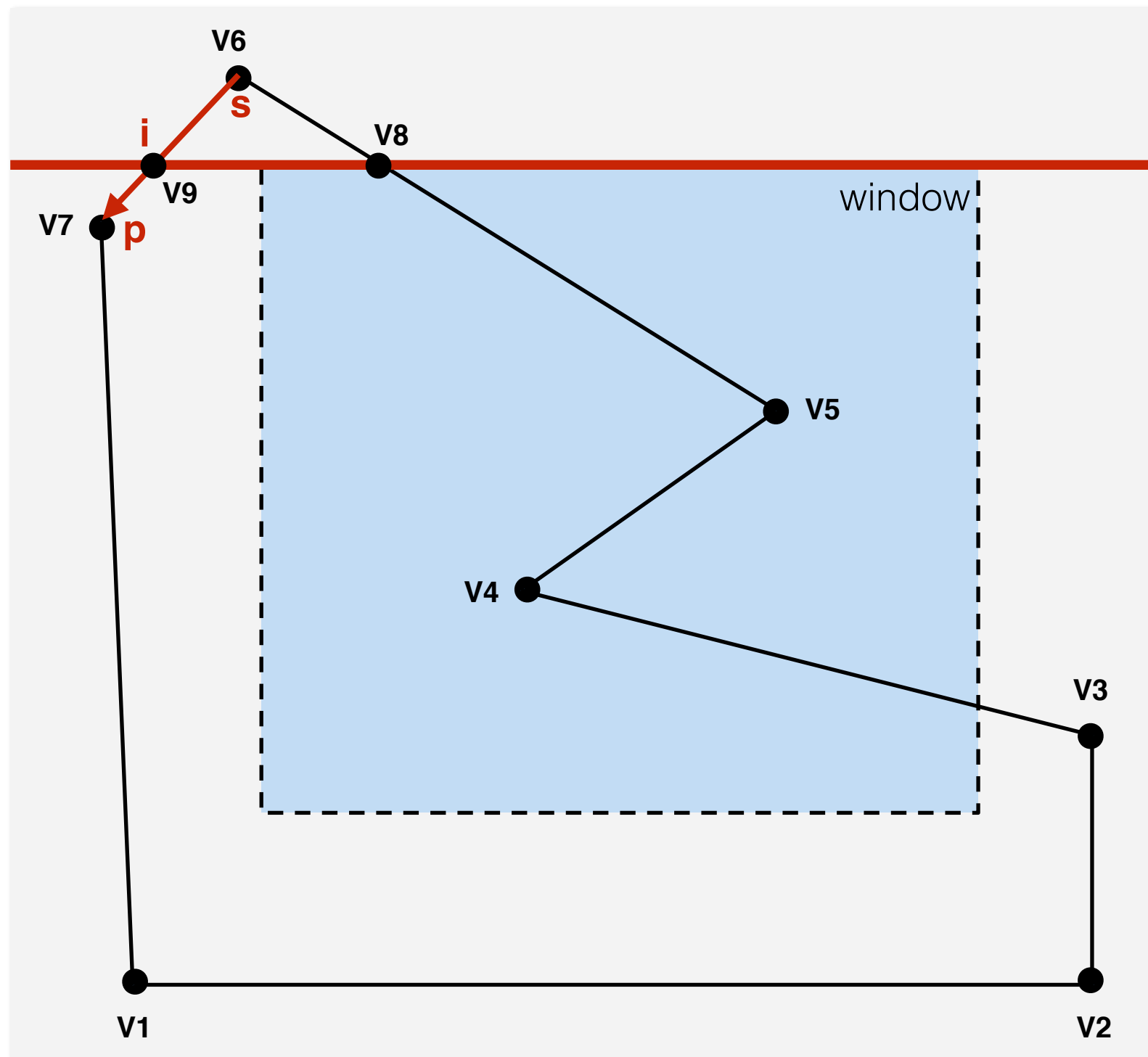
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V6, V7

outputVertices: V1, V2, V3, V4, V5, V8

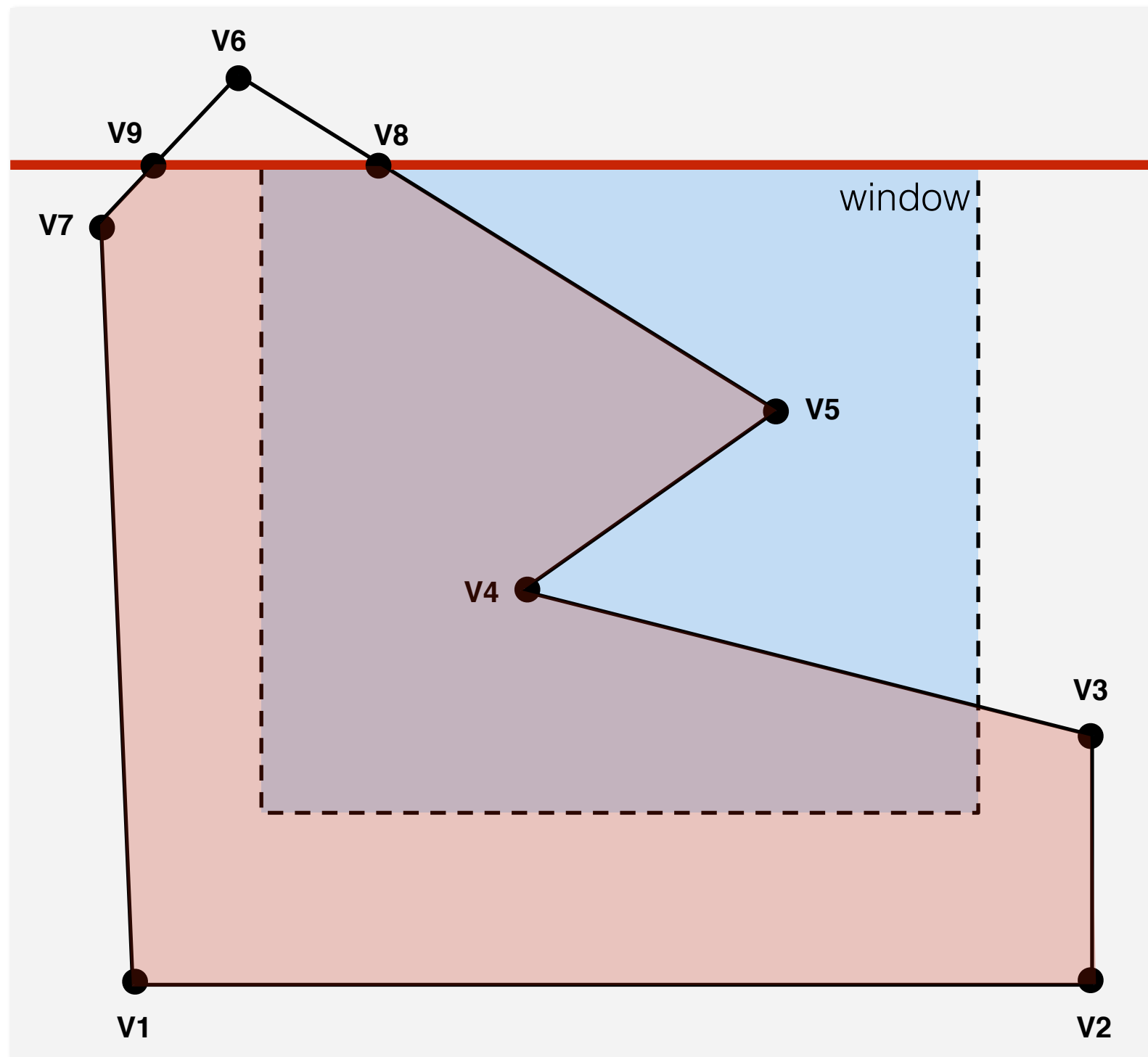
Algorithm Sutherland-Hodgman



inputVertices: $V1, V2, V3, V4, V5, V6, V7$

outputVertices: $V1, V2, V3, V4, V5, V8, V9, V7$

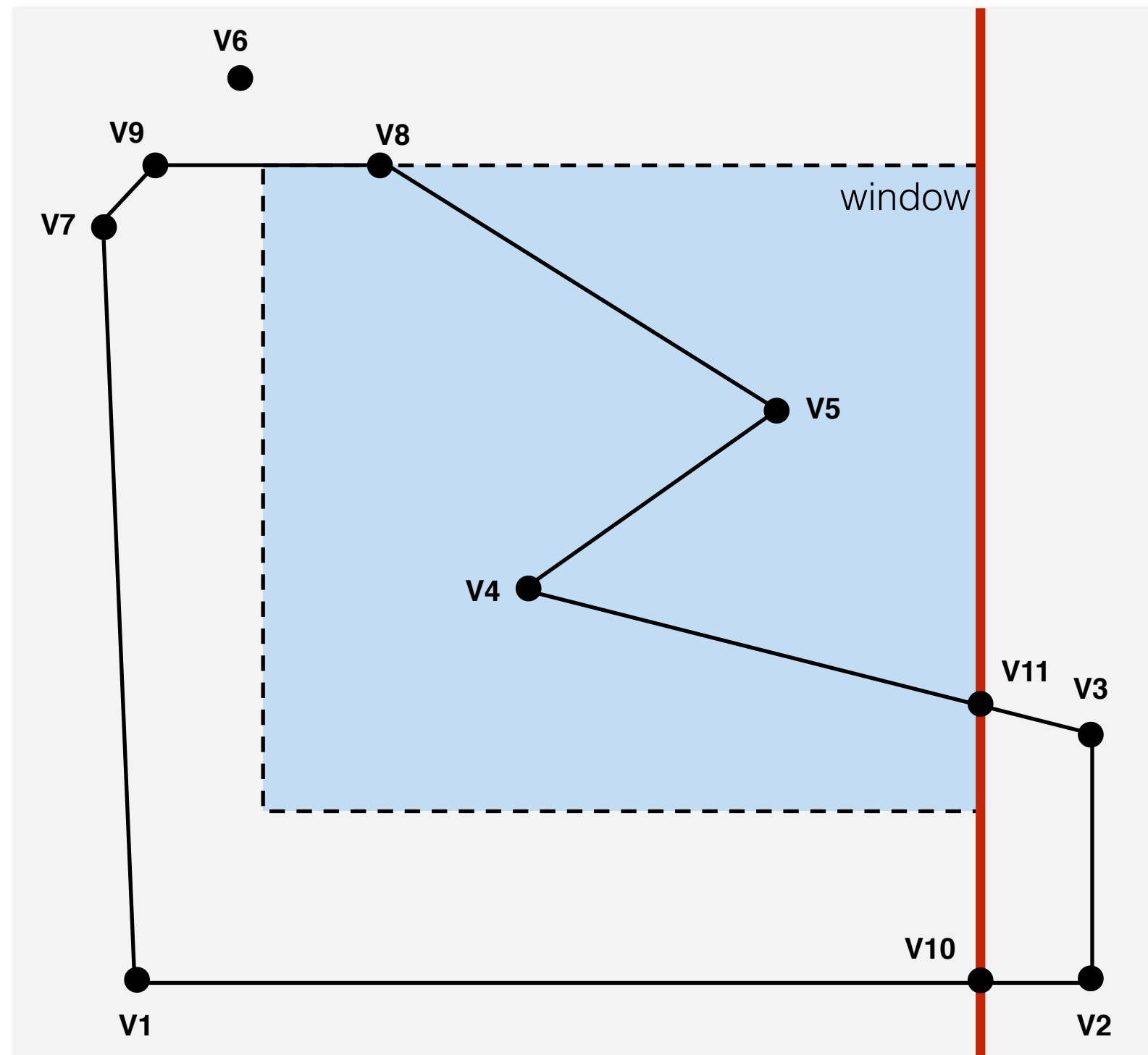
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V6, V7

outputVertices: V1, V2, V3, V4, V5, V8, V9, V7

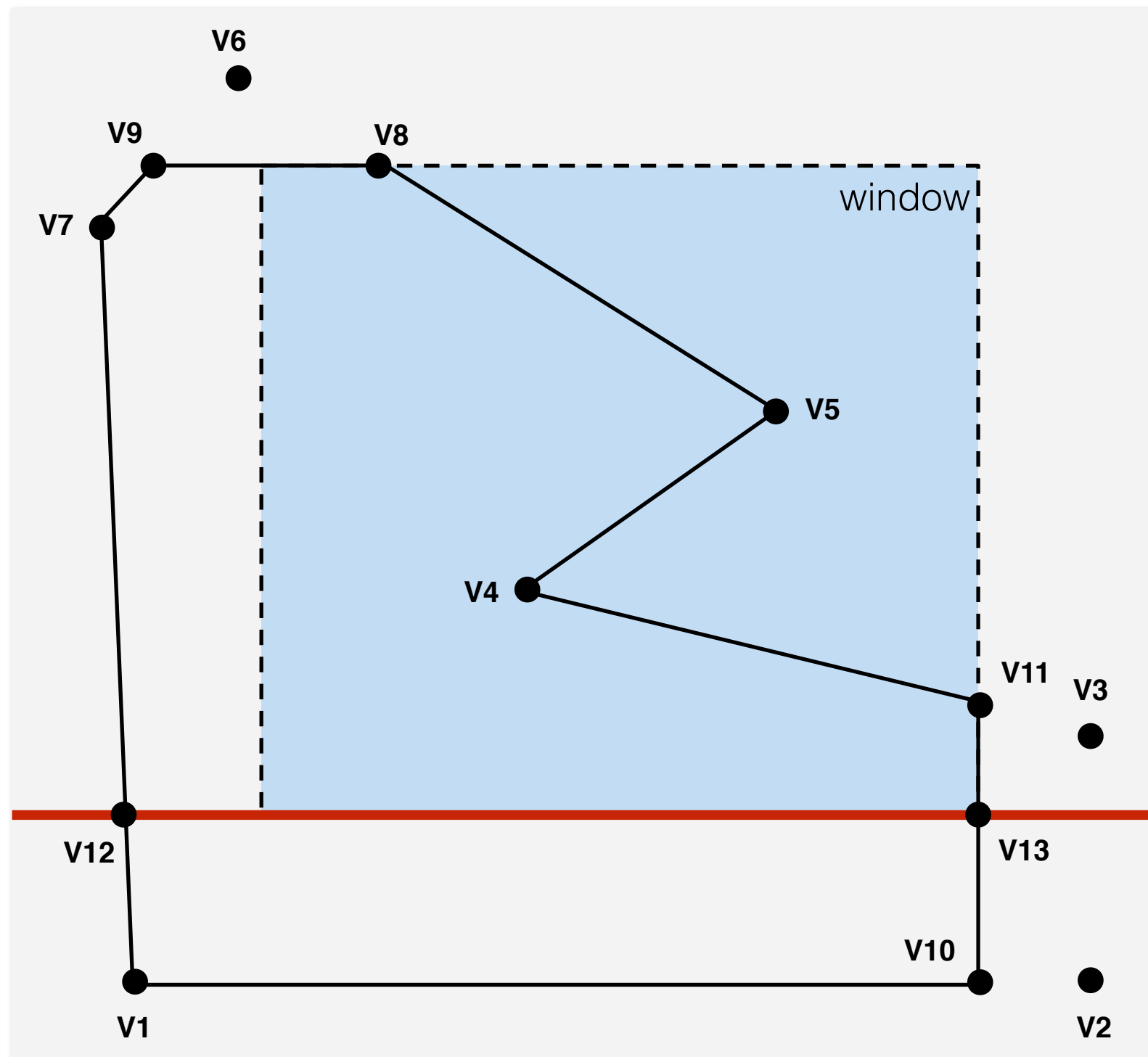
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V8, V9, V7

outputVertices: V1, V10, V11, V4, V5, V8, V9, V7

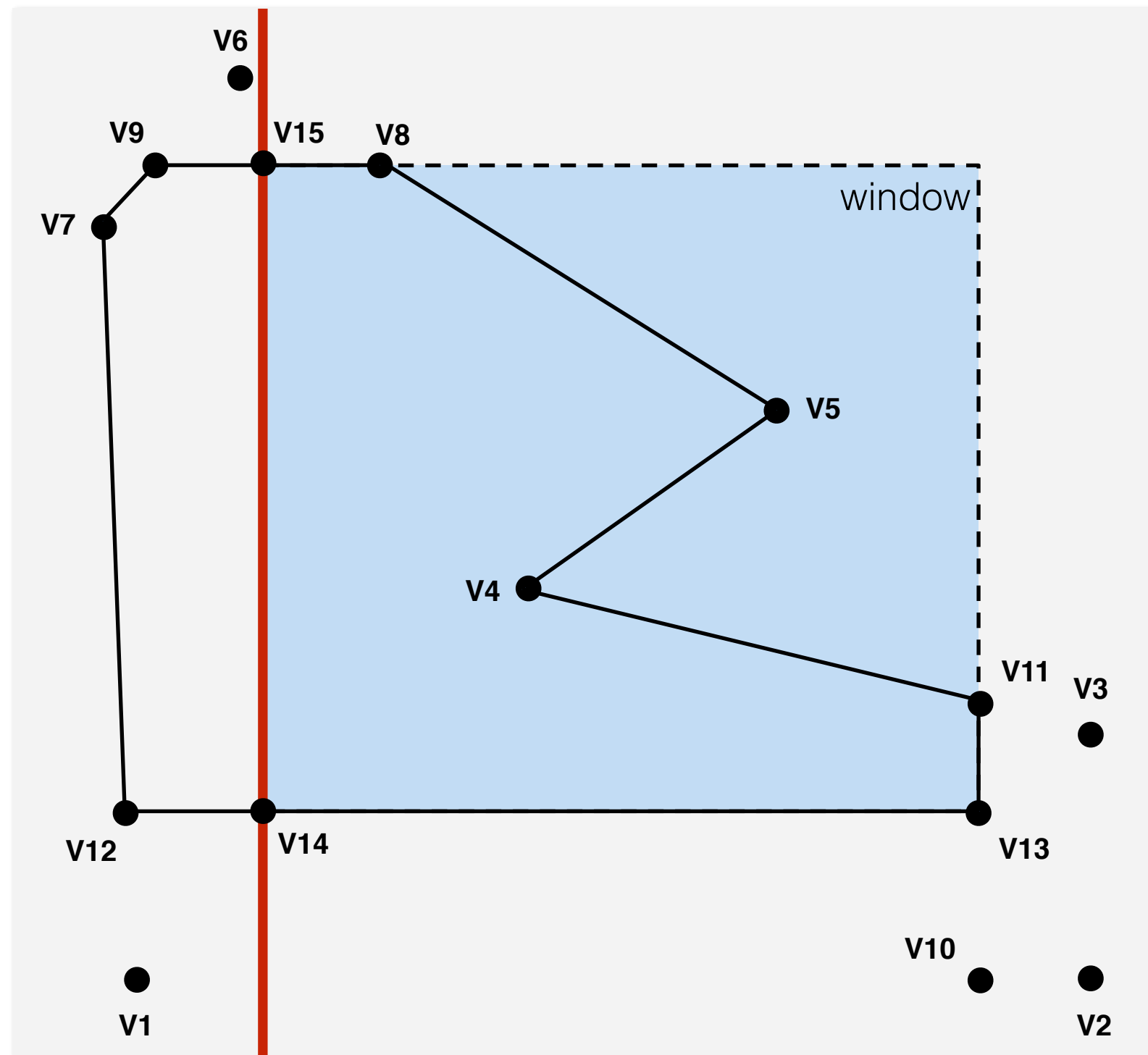
Algorithm Sutherland-Hodgman



inputVertices: V1, V10, V11, V4, V5, V8, V9, V7

outputVertices: V12, V13, V11, V4, V5, V8, V9, V7

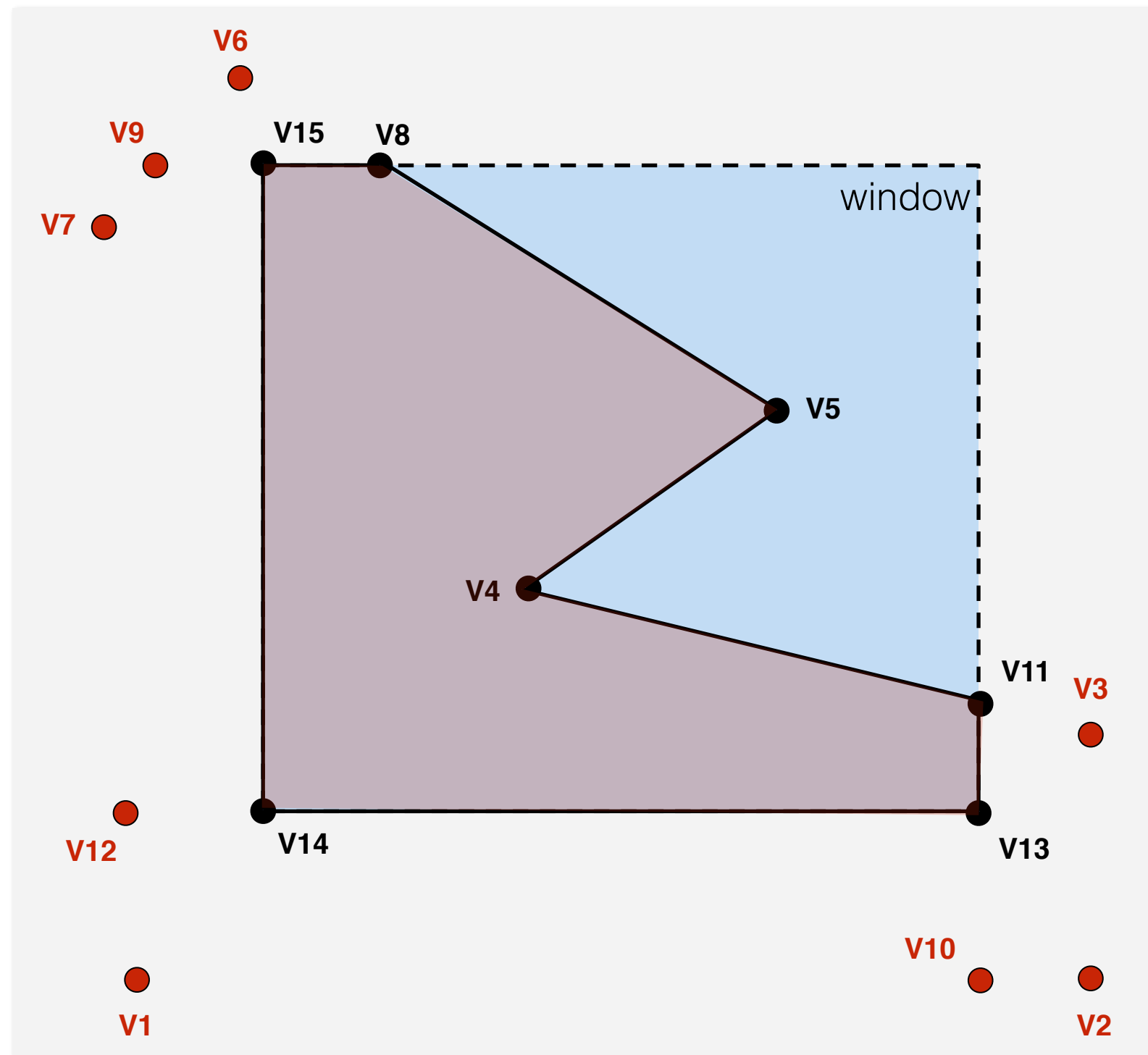
Algorithm Sutherland-Hodgman



inputVertices: V12, V13, V11, V4, V5, V8, V9, V7

outputVertices: V14, V13, V11, V4, V5, V8, V15

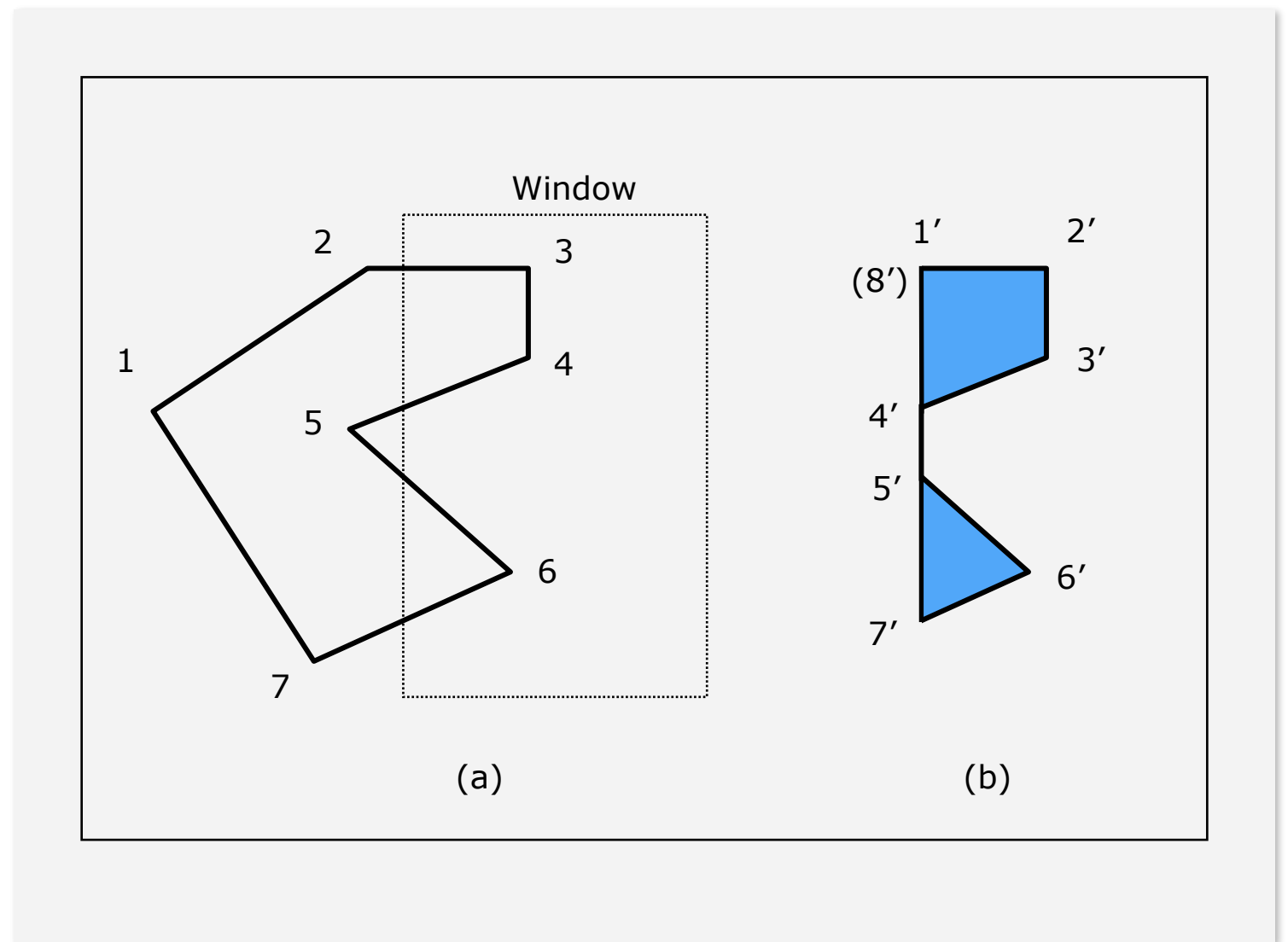
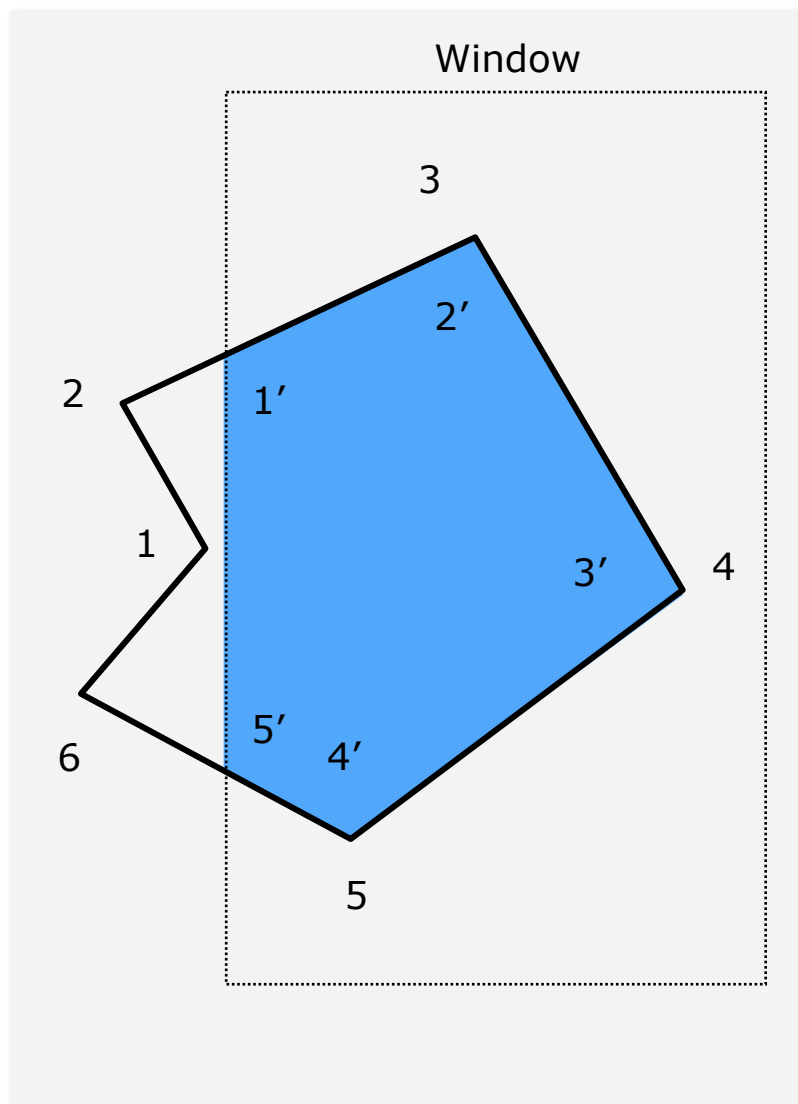
Algorithm Sutherland-Hodgman



inputVertices: V1, V2, V3, V4, V5, V6, V7

outputVertices: V14, V13, V11, V4, V5, V8, V15

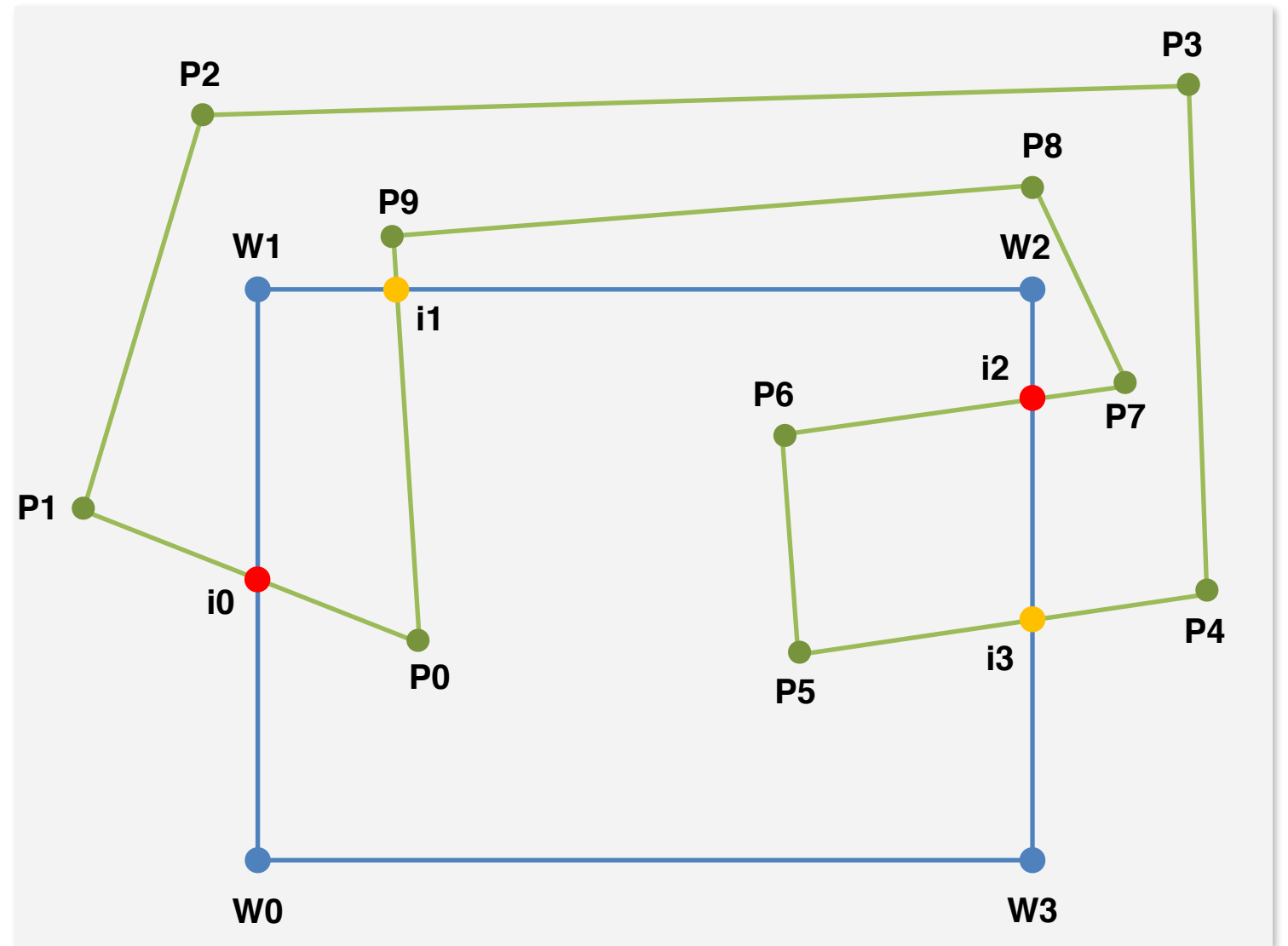
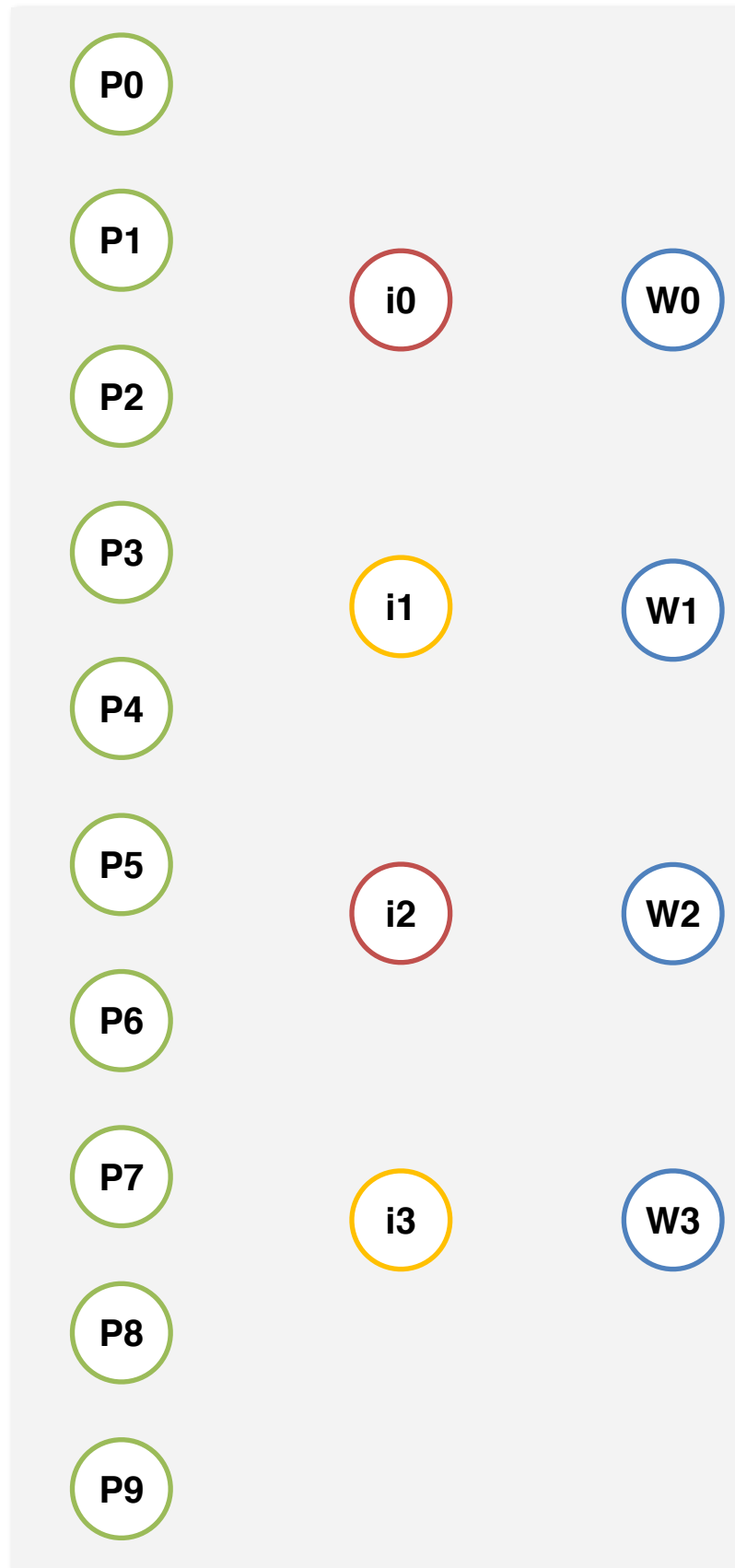
Algoritmul Sutherland-Hodgman



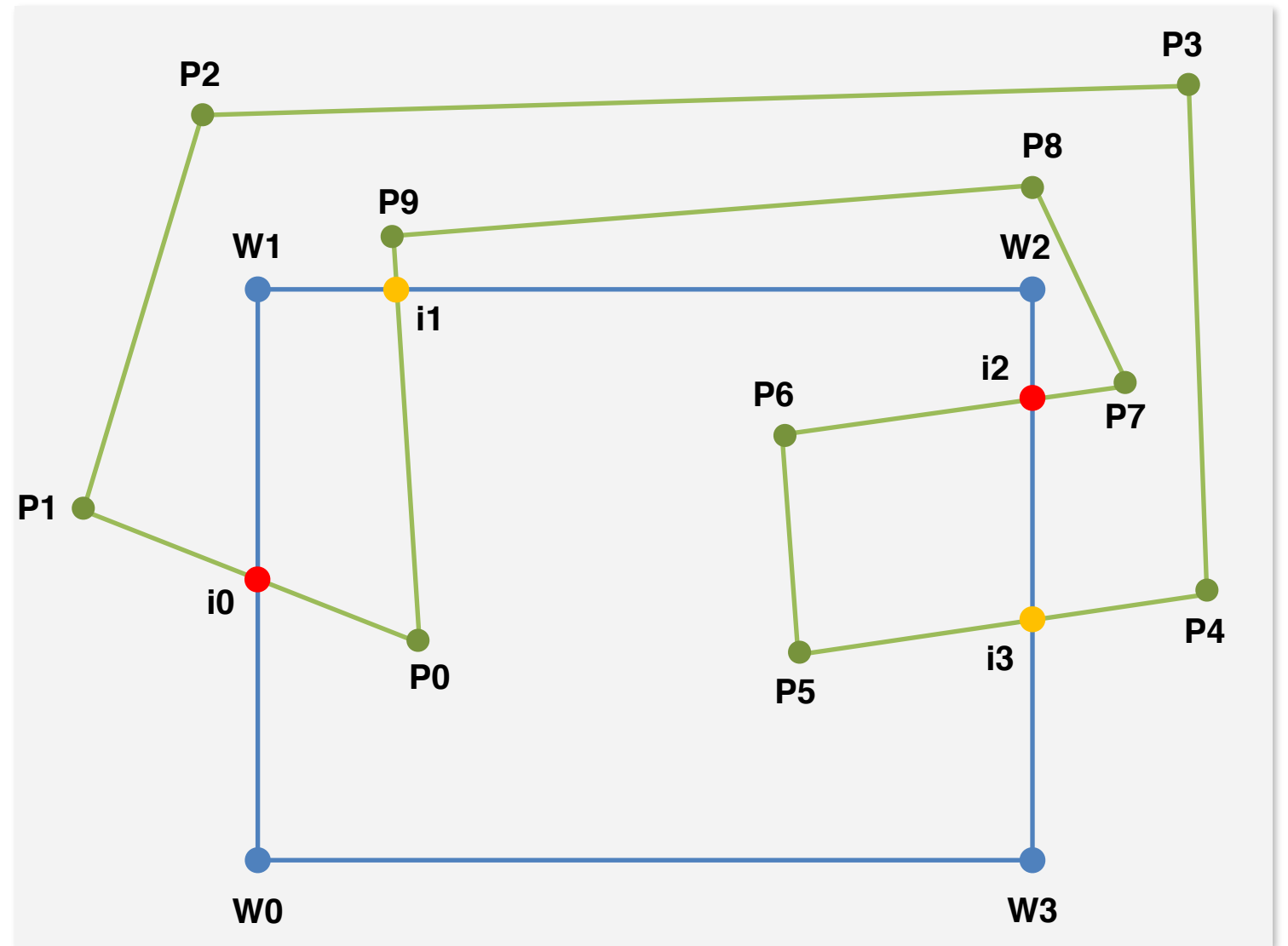
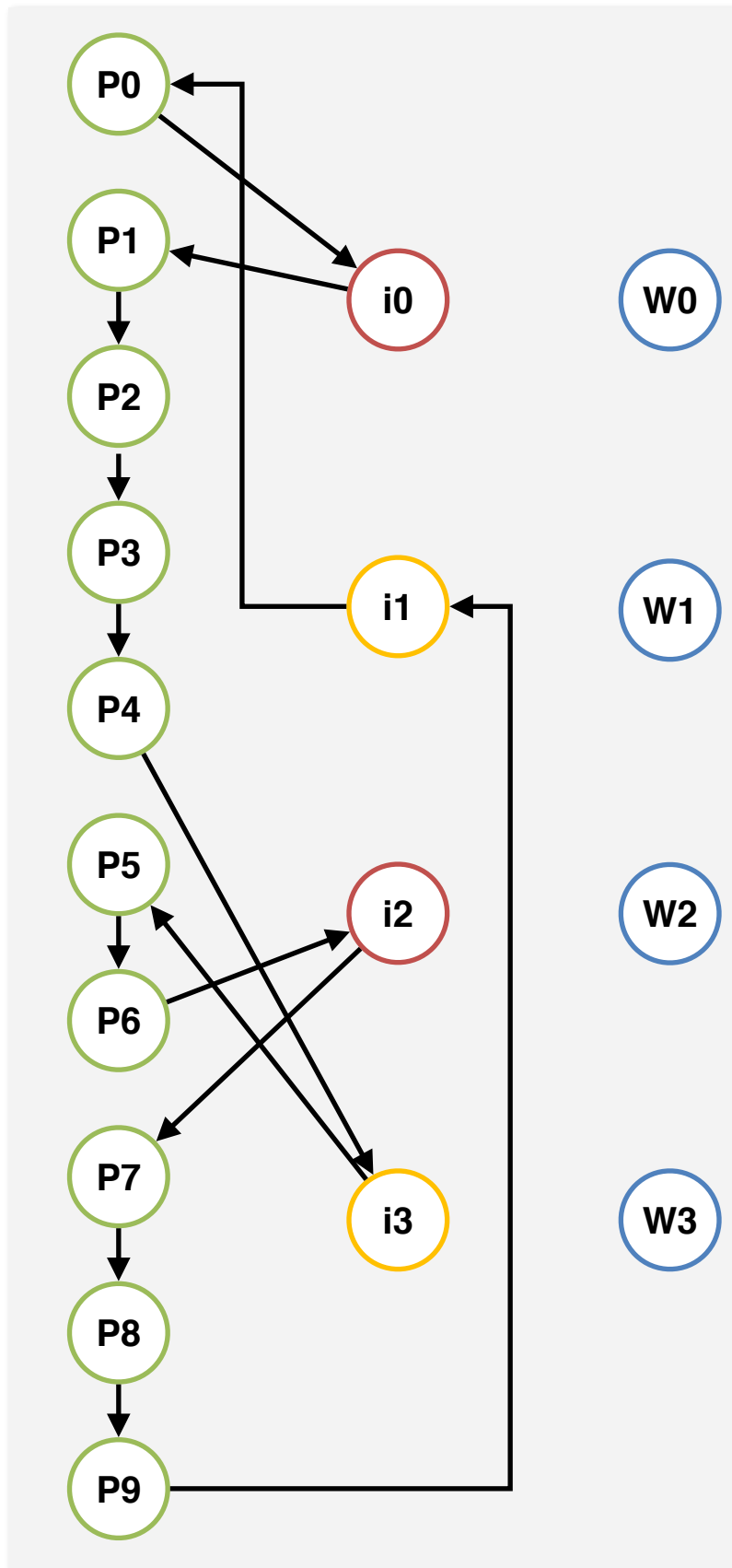
Algoritmul Weiler-Atherton

- Lista vârfurilor de intrare:
 - vârfurile poligonului de decupat
 - vârfurile poligonului față de care se efectuează decuparea
 - punctele de intersecție

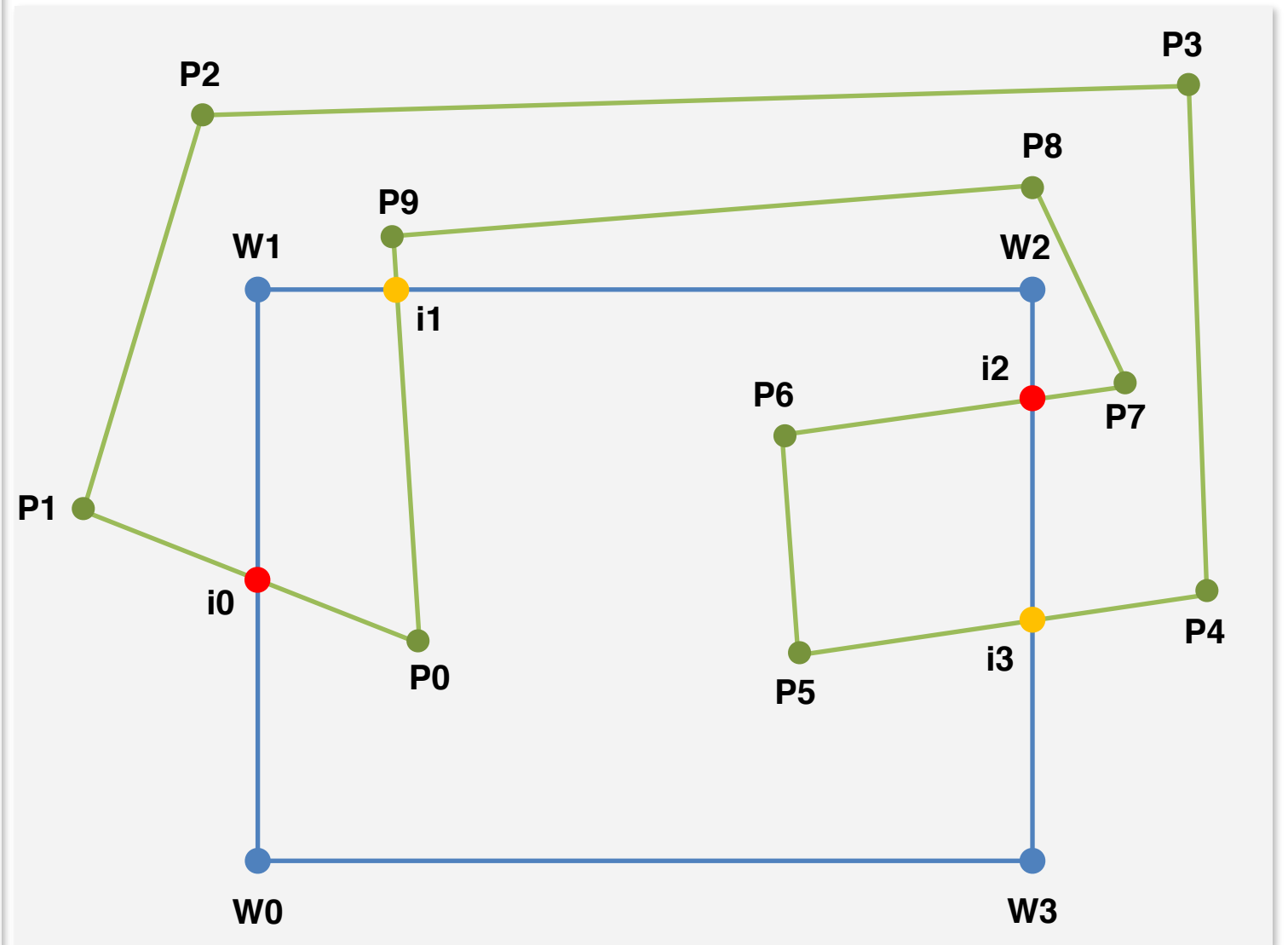
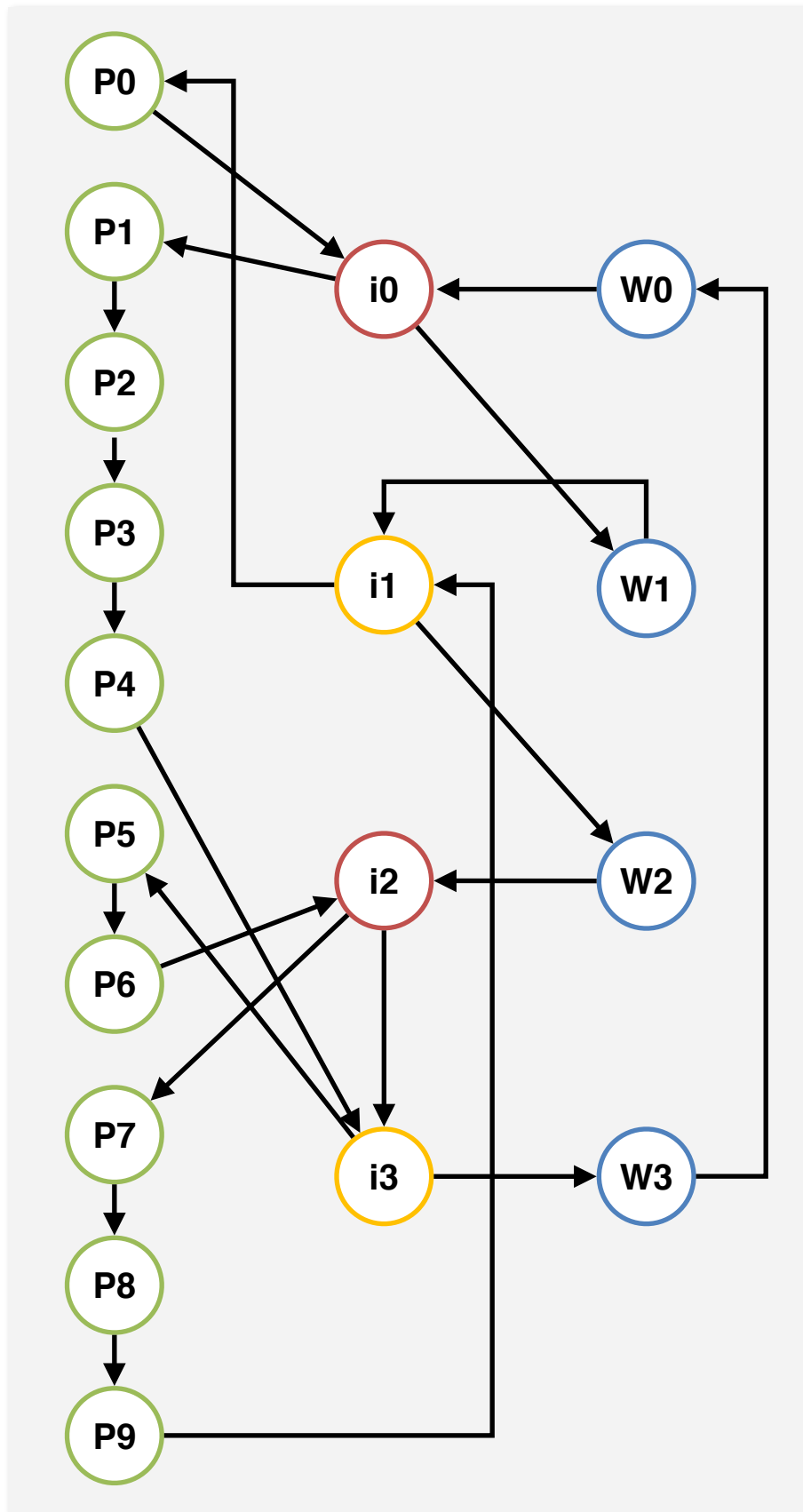
Weiler-Atherton algorithm



Weiler-Atherton algorithm



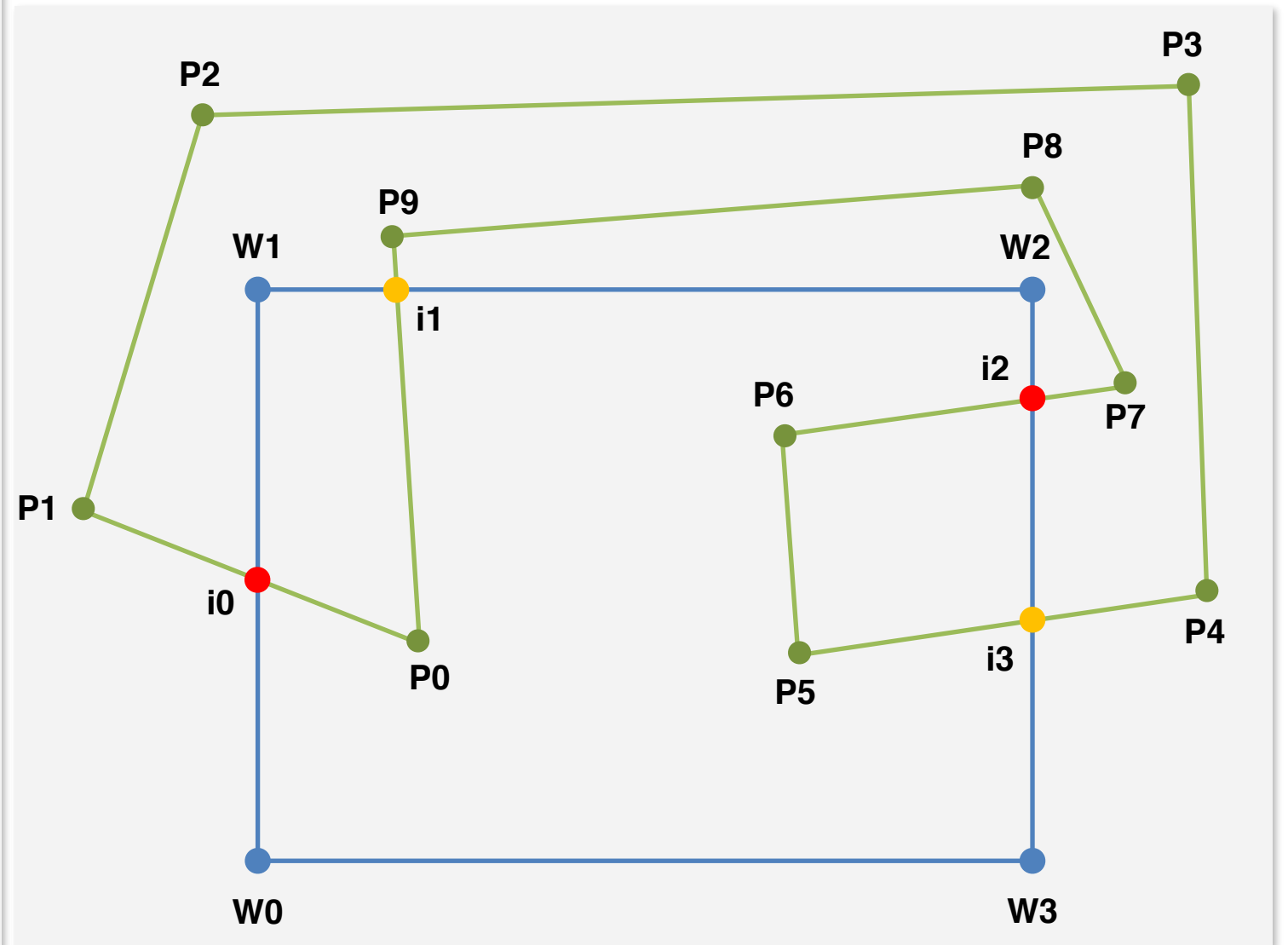
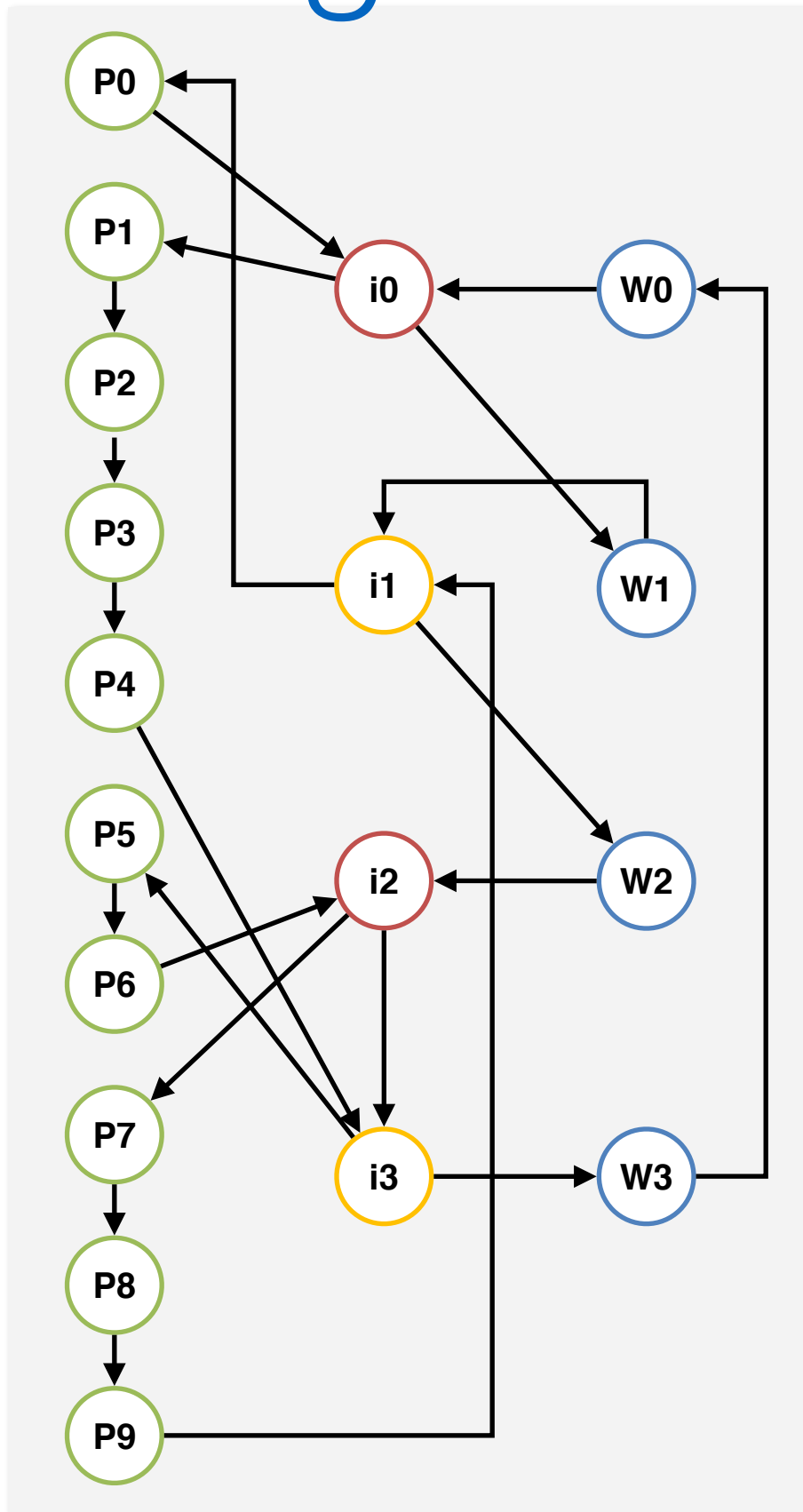
Weiler-Atherton algorithm



Algoritmul Weiler-Atherton

1. Pornim de la un punct de intersecție (ieșire) și adăugăm vârfurile de pe poligonul față de care efectuăm decuparea
2. Dacă am adăugat un punct de intersecție (intrare) adăugăm în continuare vârfuri de pe poligonul de decupat
3. Continuăm pașii 1 și 2 până în momentul în care ajungem la punctul inițial de intersecție
4. Continuăm cu următorul punct de intersecție (nevizitat) de tip ieșire parcurgând pașii 1 - 3 până nu mai avem niciun punct de intersecție nevizitat

Algorithm Weiler-Atherton



$i_0 \rightarrow W_1 \rightarrow i_1 \rightarrow P_0 \rightarrow i_0$

$i_2 \rightarrow i_3 \rightarrow P_5 \rightarrow P_6 \rightarrow i_2$